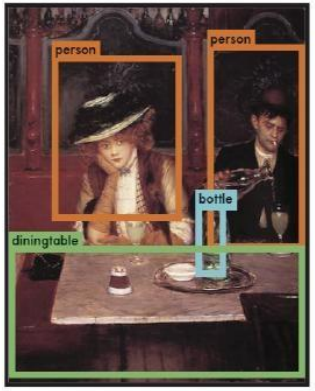




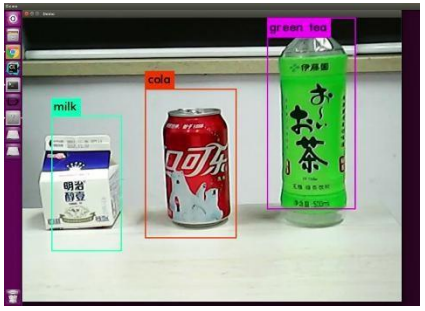


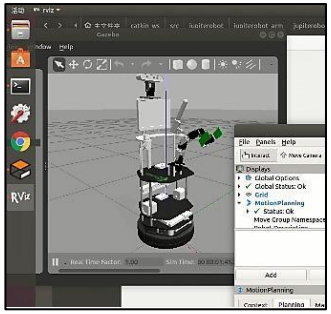
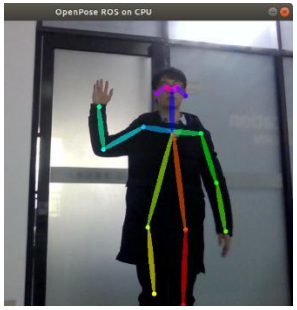


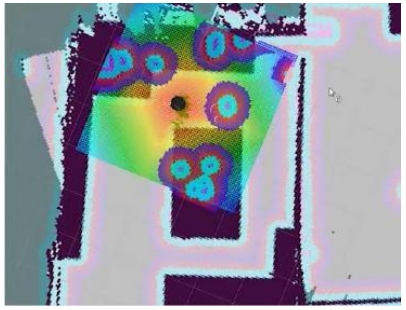
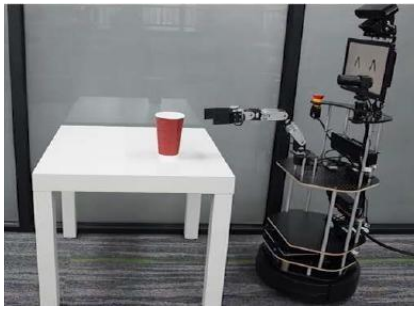


Machine Learning

PRACTICAL APPLICATIONS



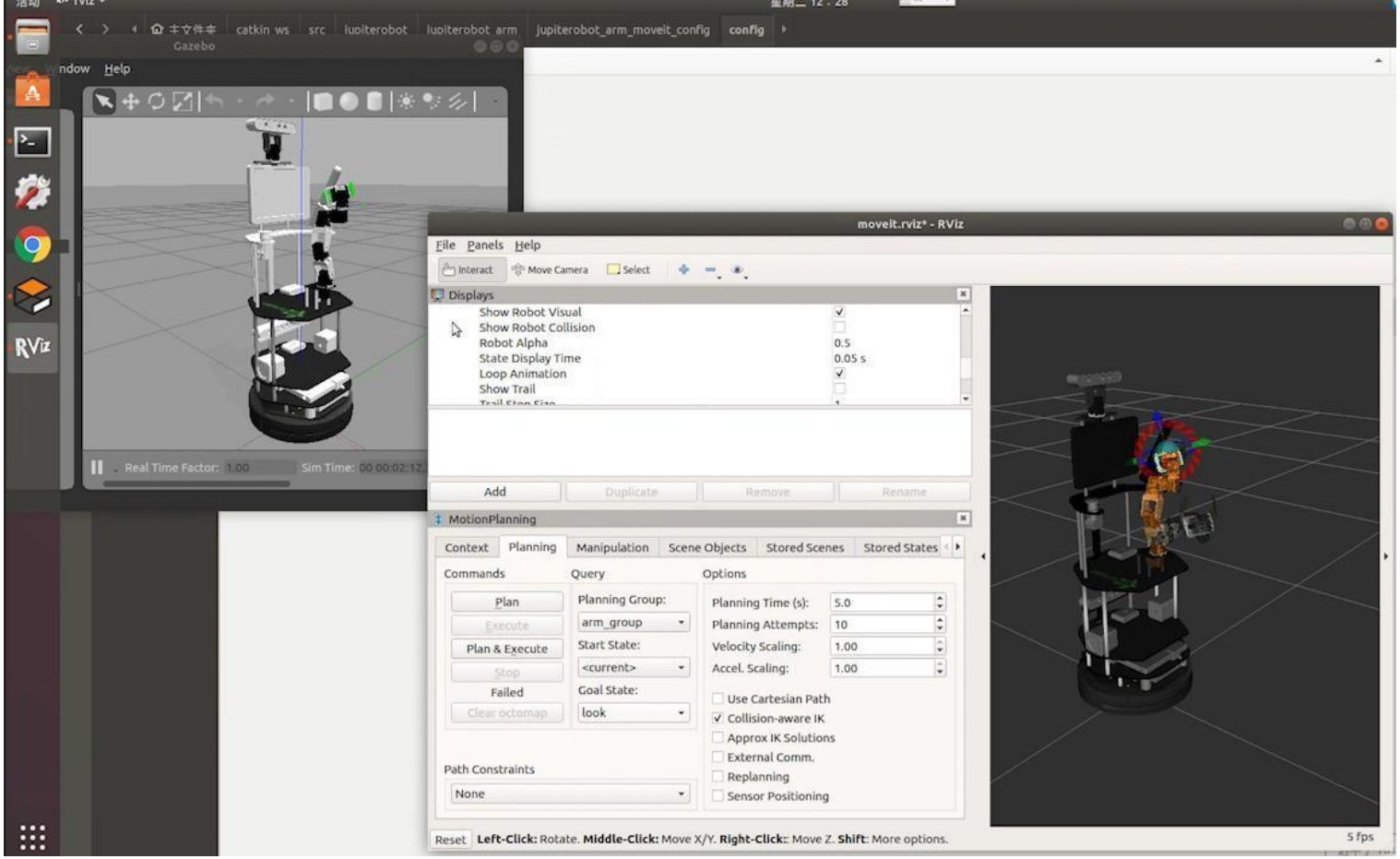



UNIVERSITY OF JINAN

08 – 10 Nov 2026 | Jeffrey Tan

Contents

1. Practical Application Development
   a. Jupiter AI Robotics System
   b. ROS Introduction
   c. Basic ROS Development
2. Natural Speech Interaction with **ML**
   a. Speech Synthesis
   b. Speech Recognition
   c. Chatbot Application
3. Computer Vision with **ML**
   a. RGB-D Sensing
   b. OpenCV Applications
   c. Deep Learning based Image Processing
   d. Model Training with YOLO
4. Mini Projects
   a. Project Presentations



PRACTICAL APPLICATION DEVELOPMENT

Jupiter AI Robotics System







Jupiter 2

Juno 2

JupiterOS 20

Jupiter AI Robotics System

http://www.jupiterobot.com/



JupiterOS 20



Jupiter 2



Robot Intelligence
AI, Machine Learning, Cloud Computing





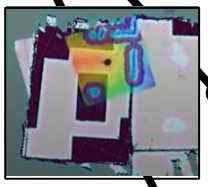
Natural Interaction
Speech Interaction



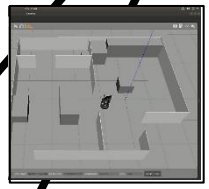


Computer Vision
Visual Perception, Object Recognition



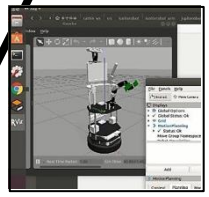


Mobile Platform
SLAM, Navigation





Robot Arm
Object Manipulation





Juno 2

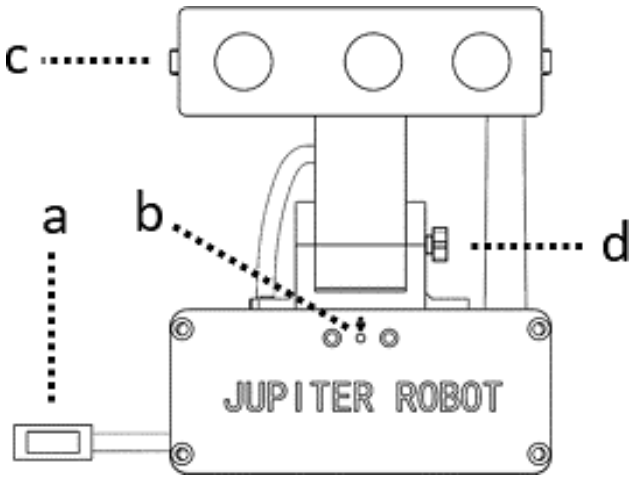
Jupiter AI Robotics System

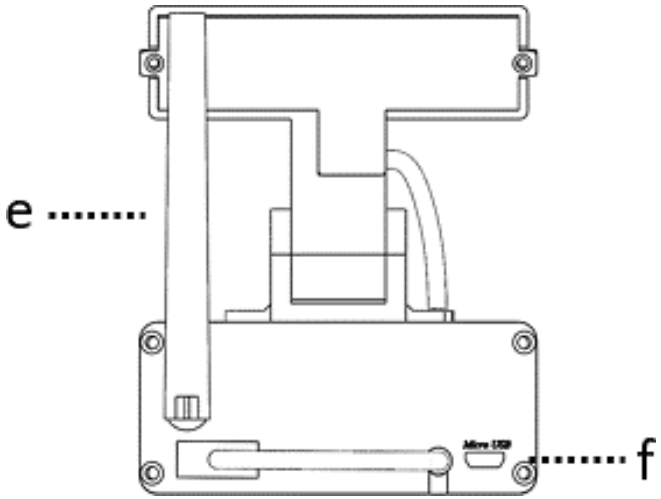
Juno

https://youtu.be/ECugI4e165k



Juno – Hardware





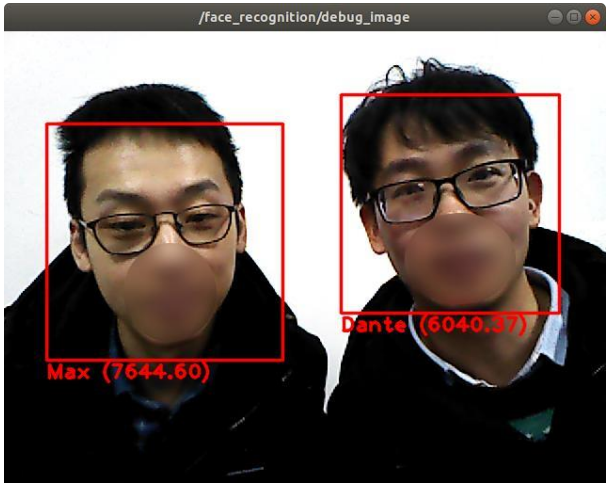


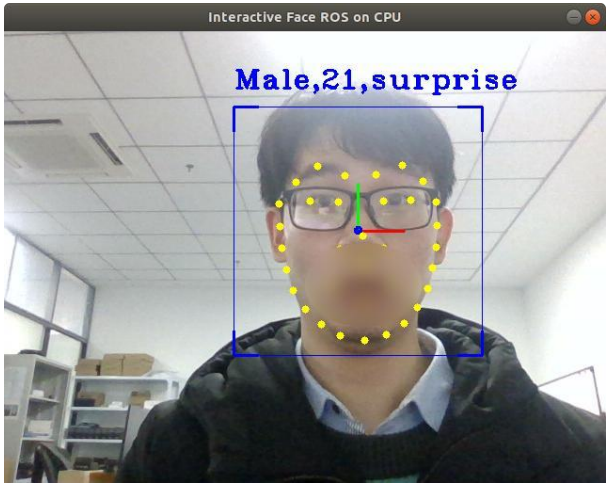
a. USB Cable
b. Microphone
c. RGB-D Camera
d. Camera Knob
e. Wi-Fi Antenna
f. Micro USB Port

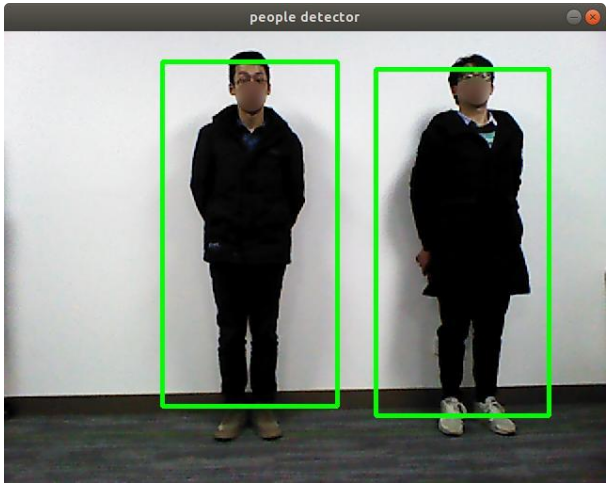
Jupiter AI Robotics System

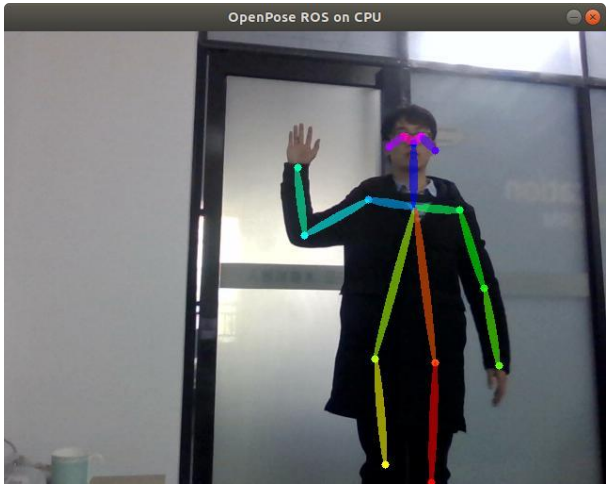
Juno

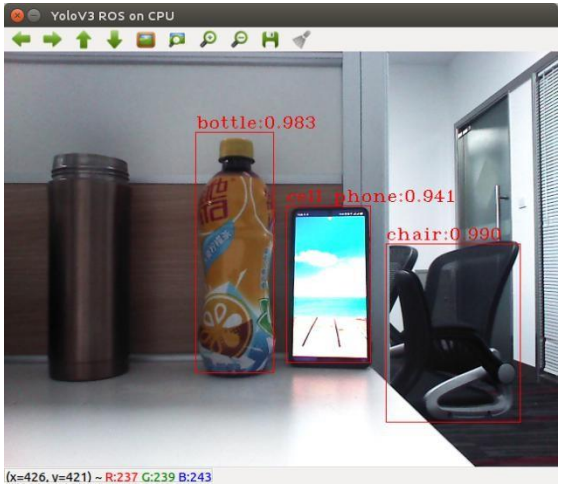
Computer Vision













Jupiter AI Robotics System

JupiterOS

Dev. Environment (Software)

- Ubuntu OS
- ROS 1 and ROS 2
- Jupiter Robot System
- Robot Model in Gazebo
- OpenCV
- YOLO
- MediaPipe
- PocketSphinx
- Etc.



Jupiter Robot Software Environment

JupiterOS 20

- Ubuntu 20.04.6 LTS (Focal Fossa)
  - https://releases.ubuntu.com/20.04/
- ROS Noetic Ninjemys [ROS 1]
  - http://wiki.ros.org/noetic
  - Tutorials: http://wiki.ros.org/ROS/Tutorials
  - Workspace: ~/catkin_ws
- ROS Foxy Fitzroy [ROS 2]
  - https://docs.ros.org/en/foxy/
  - Tutorials: https://docs.ros.org/en/foxy/Tutorials.html
  - Workspace: ~/colcon_ws
- Jupiter Robot System
  - https://github.com/jupiterobot/jupiterobot2
- RoboCup@Home Education OpenCourseWare
  - https://www.robocupathomeedu.org/learn
  - https://github.com/robocupathomeedu



Jupiter Robot – Demo GUI

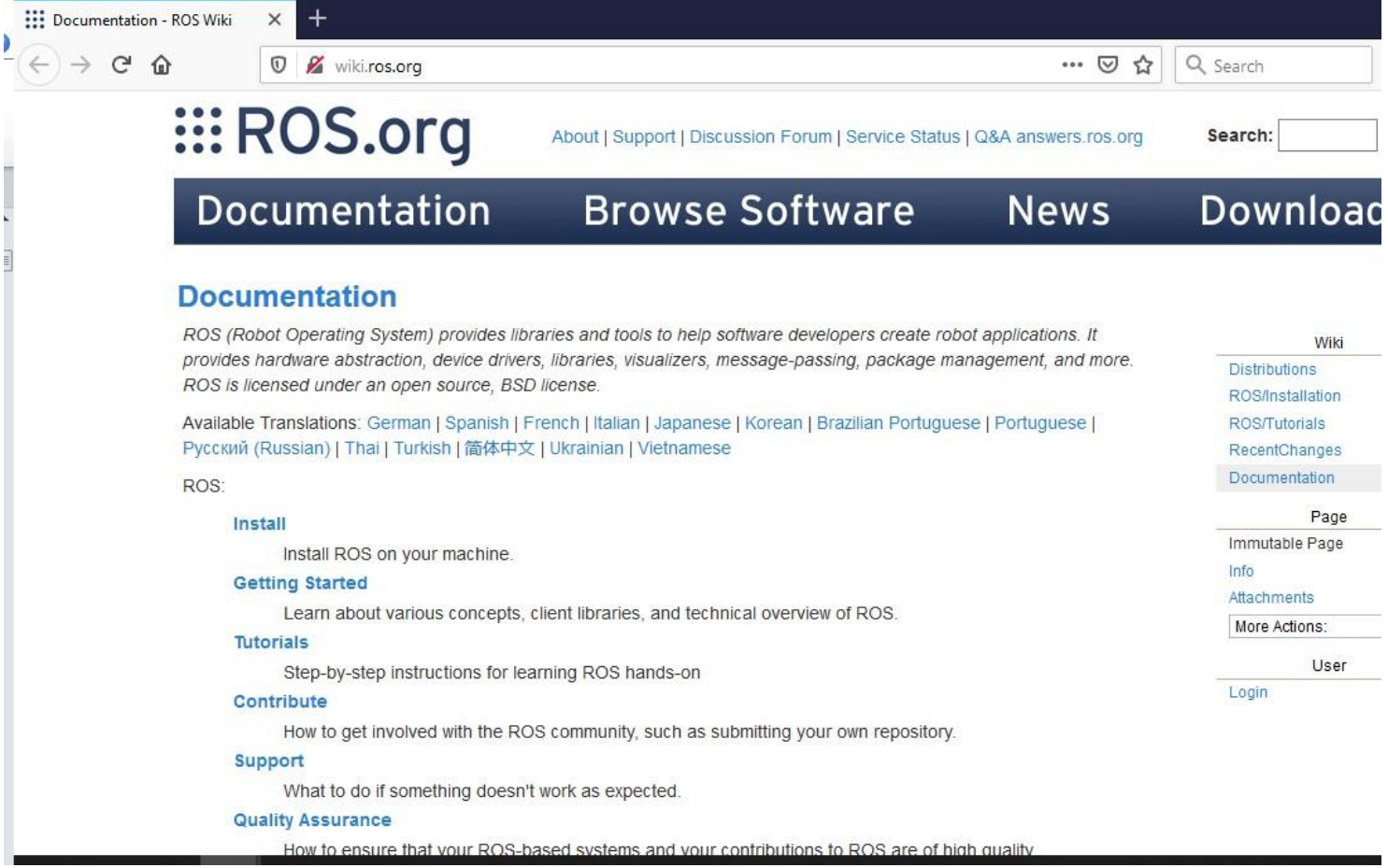
Setup

- $ cd ~/catkin_ws/src
- $ git clone https://github.com/jupiterobot/jupiterobot2_qt.git
- $ cd ~/catkin_ws
- $ catkin_make

Execution

- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch

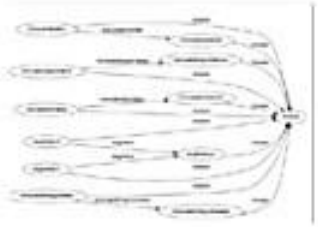




ROS INTRODUCTION

What is ROS?

ROS = Robot Operating System






ros.org

Plumbing

- Process management
- Inter-process communication
- Device drivers

Tools

- Simulation
- Visualization
- Graphical user interface
- Data logging

Capabilities

- Control
- Planning
- Perception
- Mapping
- Manipulation

Ecosystem

- Package organization
- Software distribution
- Documentation
- Tutorials

ROS Architecture & Philosophy

- **Peer to peer**
  Individual programs communicate over defined API (ROS *messages*, *services*, etc.).
- **Distributed**
  Programs can be run on multiple computers and communicate over the network.
- **Multi-lingual**
  ROS modules can be written in any language for which a client library exists (C++, Python, MATLAB, Java, etc.).
- **Light-weight**
  Stand-alone libraries are wrapped around with a thin ROS layer.
- **Free and open-source**
  Most ROS software is open-source and free to use.

ROS Master

ROS Master

- Manages the communication between nodes
- Every node registers at startup with the master



Start a master with

```
> roscore
```

More info
http://wiki.ros.org/Master

ROS Nodes

ROS Nodes

- Single-purpose, executable program
- Individually compiled, executed, and managed
- Organized in *packages*

Run a node with

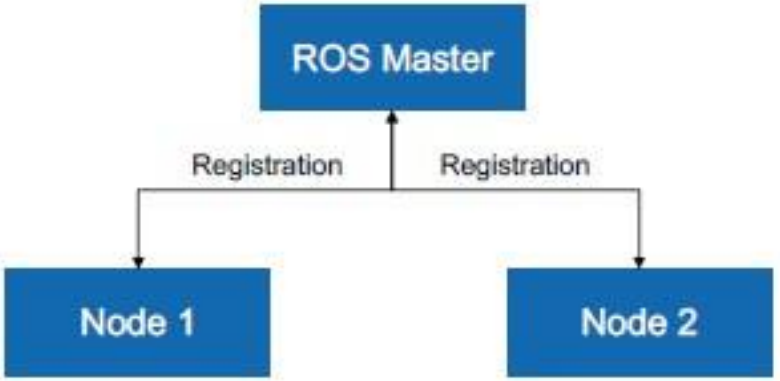
```
> rosrun package_name node_name
```

See active nodes with

```
> rosnode list
```

Retrieve information about a node with

```
> rosnode info node_name
```



More info
http://wiki.ros.org/rosnode

ROS Topics

ROS Topics

- Nodes communicate over *topics*
  - Nodes can *publish* or *subscribe* to a topic
  - Typically, 1 publisher and *n* subscribers
- Topic is a name for a stream of *messages*

List active topics with

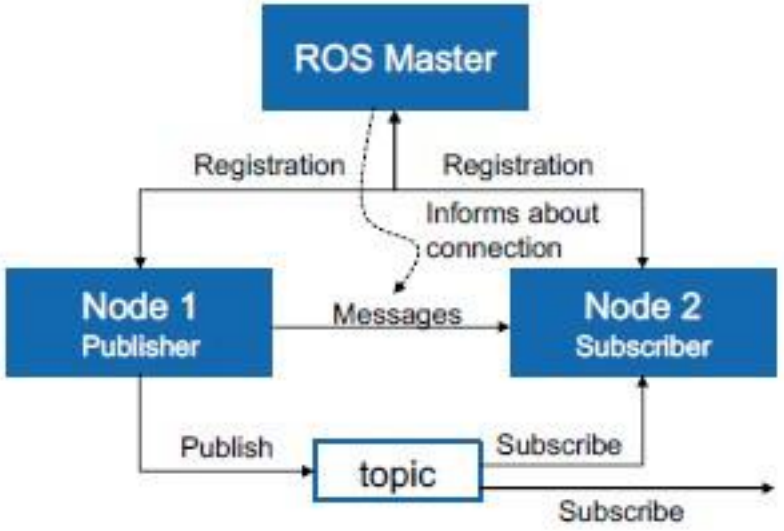
```
> rostopic list
```

Subscribe and print the contents of a topic with

```
> rostopic echo /topic
```

Show information about a topic with

```
> rostopic info /topic
```



More info
http://wiki.ros.org/rostopic

ROS Messages

ROS Messages

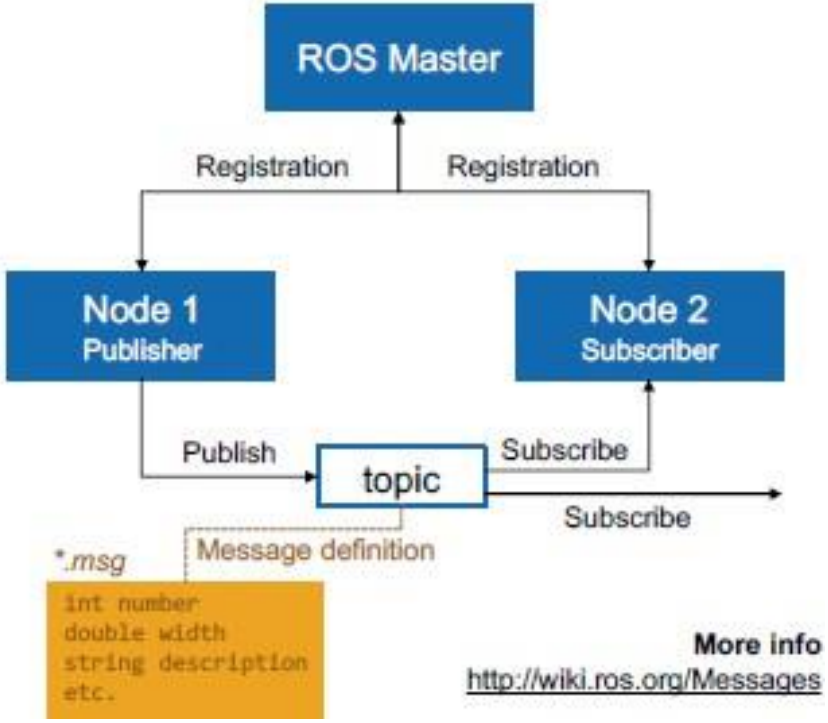
- Data structure defining the *type* of a topic
- Compromised of a nested structure of integers, floats, booleans, strings etc. and arrays of objects
- Defined in **.msg* files

See the type of a topic

```
> rostopic type /topic
```

Publish a message to a topic

```
> rostopic pub /topic type args
```



ROS Messages

ROS Messages

Pose Stamped Example

geometry_msgs/Point.msg

```
float64 x
float64 y
float64 z
```

sensor_msgs/Image.msg

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
uint32 height
uint32 width
string encoding
uint8 is_bigendian
uint32 step
uint8[] data
```

geometry_msgs/PoseStamped.msg

```
std_msgs/Header header
 uint32 seq
 time stamp
 string frame_id
geometry_msgs/Pose pose
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Quaternion orientation
    float64 x
    float64 y
    float64 z
    float64 w
```

ROS Launch

ROS Launch

- *launch* is a tool for launching multiple nodes (as well as setting parameters)
- Are written in XML as **.launch* files
- If not yet running, launch automatically starts a roscore

Browse to the folder and start a launch file with

```
> roslaunch file_name.launch
```

Start a launch file from a package with

```
> roslaunch package_name file_name.launch
```

More info
http://wiki.ros.org/roslaunch

Example console output for
`roslaunch roscpp_tutorials talker_listener.launch`

```
student@ubuntu:~/catkin_ws$ roslaunch roscpp_tutorials talker_listener.launch
... logging to /home/student/.ros/log/794321aa-e950-11e6-95db-000c297bd368/rosl
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:37592/

SUMMARY
========

PARAMETERS
 * /rosdistro: indigo
 * /rosversion: 1.11.20

NODES
  /
    listener (roscpp_tutorials/listener)
    talker (roscpp_tutorials/talker)

auto-starting new master
process[master]: started with pid [5772]
ROS_MASTER_URI=http://localhost:11311

setting /run_id to 794321aa-e950-11e6-95db-000c297bd368
process[rosout-1]: started with pid [5785]
started core service [/rosout]
process[listener-2]: started with pid [5788]
process[talker-3]: started with pid [5795]
[ INFO] [1486044252.537801350]: hello world 0
[ INFO] [1486044252.638886504]: hello world 1
[ INFO] [1486044252.738279674]: hello world 2
[ INFO] [1486044252.838357245]: hello world 3
```

ROS Launch

ROS Launch
File Structure

! Attention when copy & pasting code from the internet

talker_listener.launch

```
<launch>
    <node name="listener" pkg="roscpp_tutorials" type="listener" output="screen"/>
    <node name="talker" pkg="roscpp_tutorials" type="talker" output="screen"/>
</launch>
```

! Notice the syntax difference for self-closing tags: `<tag></tag>` and `<tag/>`

- **launch**: Root element of the launch file
- **node**: Each *<node>* tag specifies a node to be launched
- **name**: Name of the node (free to choose)
- **pkg**: Package containing the node
- **type**: Type of the node, there must be a corresponding executable with the same name
- **output**: Specifies where to output log messages (`screen`: console, `log`: log file)

More info
http://wiki.ros.org/roslaunch/XML
http://wiki.ros.org/roslaunch/Tutorials/Roslaunch%20tips%20for%20larger%20projects

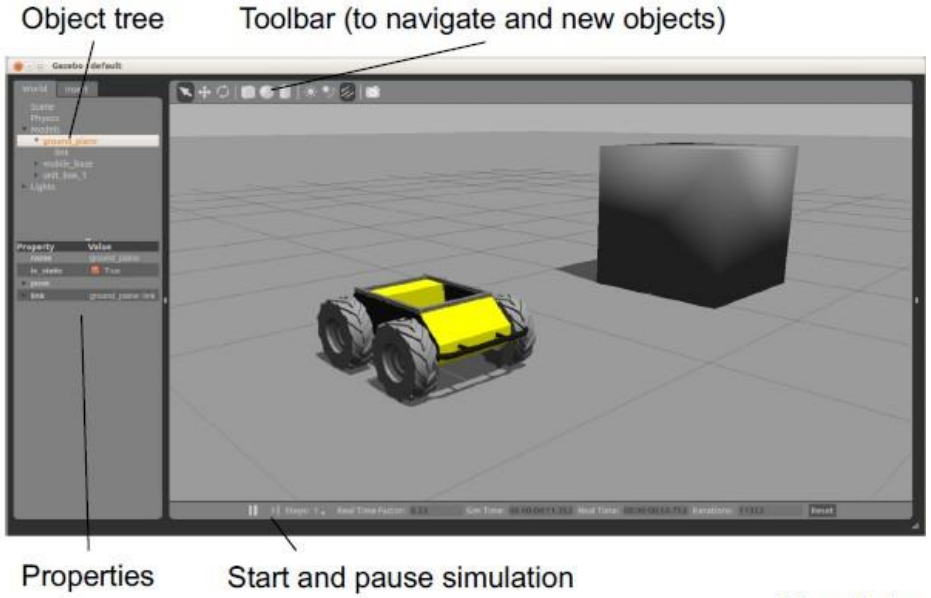
Gazebo Simulator

Gazebo Simulator

- Simulate 3d rigid-body dynamics
- Simulate a variety of sensors including noise
- 3d visualization and user interaction
- Includes a database of many robots and environments (*Gazebo worlds*)
- Provides a ROS interface
- Extensible with plugins

Run Gazebo with

```
> rosrun gazebo_ros gazebo
```



More info

http://gazebosim.org/

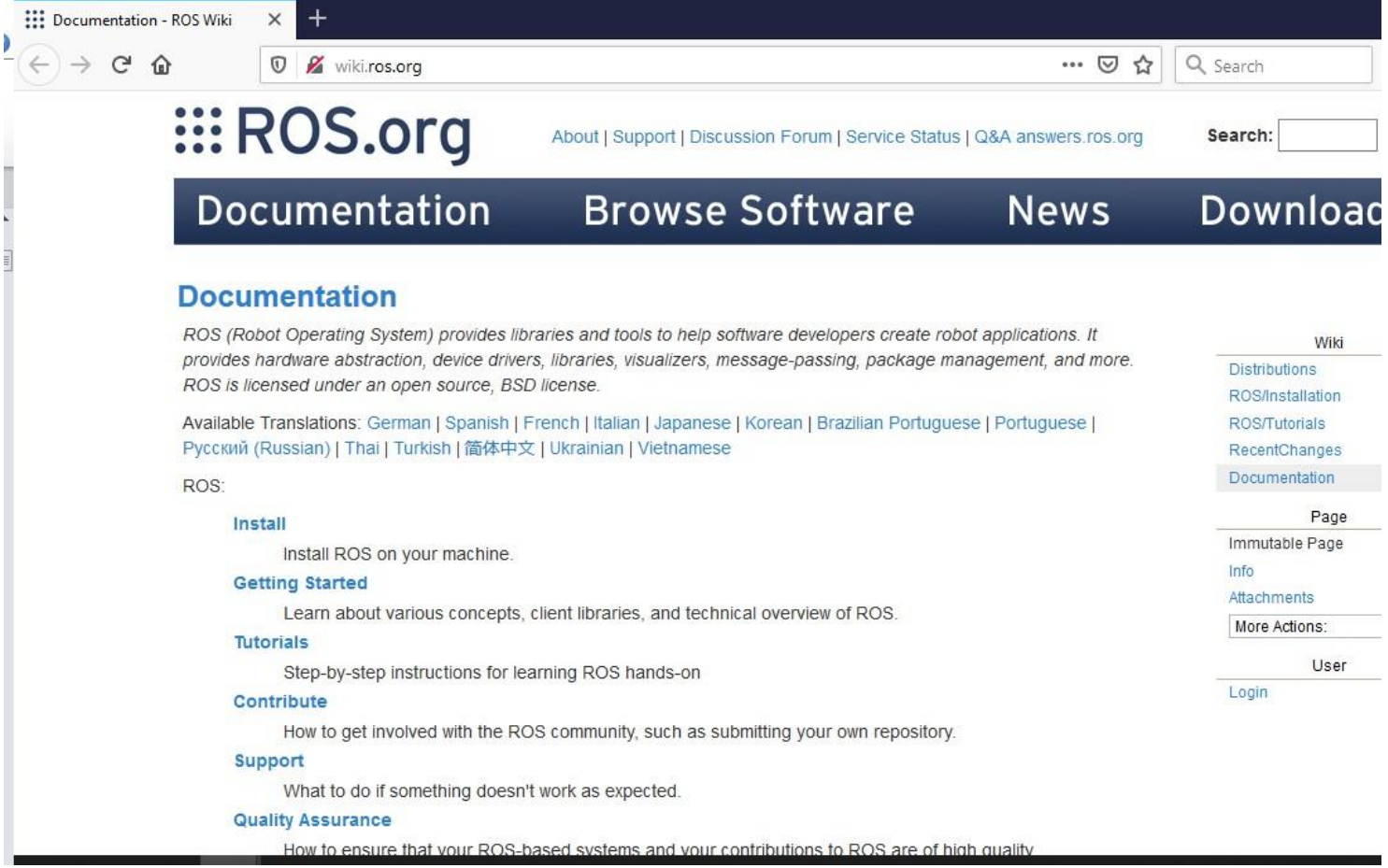
http://gazebosim.org/tutorials

Other Important ROS Features

- ROS package structure
- ROS C++ client library (roscpp)
- RViz visualization
- TF Transformation System
- rqt User Interface
- Robot models (URDF)
- Simulation descriptions (SDF)
- ROS time
- ROS bags

Other Important ROS Features

- ROS publishers and subscribers
- ROS parameter server
- ROS services
- ROS actions (actionlib)



BASIC ROS DEVELOPMENT

Roscore

Example

Console Tab Nr. 1 – Starting a *roscore*

Start a roscore with

```
> roscore
```

```
student@ubuntu:~/catkin_ws$ roscore
... logging to /home/student/.ros/log/6c1852aa-e961-11e6-8543-000c297bd368/ros
launch-ubuntu-6696.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:34089/
ros_comm version 1.11.20


SUMMARY
========

PARAMETERS
 * /rosdistro: indigo
 * /rosversion: 1.11.20

NODES

auto-starting new master
process[master]: started with pid [6708]
ROS_MASTER_URI=http://ubuntu:11311/

setting /run_id to 6c1852aa-e961-11e6-8543-000c297bd368
process[rosout-1]: started with pid [6721]
started core service [/rosout]
```

Talker Node

Example

Console Tab Nr. 2 – Starting a *talker* node

Run a talker demo node with

```
> rosrun roscpp_tutorials talker
```

```
student@ubuntu:~/catkin_ws$ rosrun roscpp_tutorials talker
[ INFO] [1486051708.424661519]: hello world 0
[ INFO] [1486051708.525227845]: hello world 1
[ INFO] [1486051708.624747612]: hello world 2
[ INFO] [1486051708.724826782]: hello world 3
[ INFO] [1486051708.825928577]: hello world 4
[ INFO] [1486051708.925379775]: hello world 5
[ INFO] [1486051709.024971132]: hello world 6
[ INFO] [1486051709.125450960]: hello world 7
[ INFO] [1486051709.225272747]: hello world 8
[ INFO] [1486051709.325389210]: hello world 9
```

Analyze Talker Node

Example

Console Tab Nr. 3 – Analyze *talker* node

See the list of active nodes

```
> rosnode list
```

```
student@ubuntu:~/catkin_ws$ rosnode list
/rosout
/talker
```

Show information about the *talker* node

```
> rosnode info /talker
```

```
student@ubuntu:~/catkin_ws$ rosnode info /talker
--------------------------------------------------------------------------------
Node [/talker]
Publications:
 * /chatter [std_msgs/String]
 * /rosout [rosgraph_msgs/Log]

Subscriptions: None

Services:
 * /talker/get_loggers
 * /talker/set_logger_level
```

See information about the *chatter* topic

```
> rostopic info /chatter
```

```
student@ubuntu:~/catkin_ws$ rostopic info /chatter
Type: std_msgs/String

Publishers:
 * /talker (http://ubuntu:39173/)

Subscribers: None
```

Analyze Chatter Topic

Example

Console Tab Nr. 3 – Analyze *chatter* topic

Check the type of the *chatter* topic

```
> rostopic type /chatter
```

```
student@ubuntu:~/catkin_ws$ rostopic type /chatter
std_msgs/String
```

Show the message contents of the topic

```
> rostopic echo /chatter
```

```
student@ubuntu:~/catkin_ws$ rostopic echo /chatter
data: hello world 11874
---
data: hello world 11875
---
data: hello world 11876
```

Analyze the frequency

```
> rostopic hz /chatter
```

```
student@ubuntu:~/catkin_ws$ rostopic hz /chatter
subscribed to [/chatter]
average rate: 9.991
        min: 0.099s max: 0.101s std dev: 0.00076s window: 10
average rate: 9.996
        min: 0.099s max: 0.101s std dev: 0.00069s window: 20
```

Listener Node

Example

Console Tab Nr. 4 – Starting a *listener* node

Run a listener demo node with

```
> rosrun roscpp_tutorials listener
```

```
student@ubuntu:~/catkin_ws$ rosrun roscpp_tutorials listener
[ INFO] [1486053802.204104598]: I heard: [hello world 19548]
[ INFO] [1486053802.304538827]: I heard: [hello world 19549]
[ INFO] [1486053802.403853395]: I heard: [hello world 19550]
[ INFO] [1486053802.504438133]: I heard: [hello world 19551]
[ INFO] [1486053802.604297608]: I heard: [hello world 19552]
```

Analyze Chatter Topic Again

Example

Console Tab Nr. 3 – Analyze

See the new *listener* node with

```
> rosnode list
```



Show the connection of the nodes over the chatter topic with

```
> rostopic info /chatter
```



Try: $ rqt_graph

Publish Message from Terminal

Example

Console Tab Nr. 3 – Publish Message from Console

Close the *talker* node in console nr. 2 with `Ctrl + C`

Publish your own message with

```
> rostopic pub /chatter std_msgs/String
  "data: '                                '"
```

Insert Own Text

```
student@ubuntu:~/catkin_ws$ rostopic pub /chatter std_msgs/String "data: 'ETH
Zurich ROS Course'"
publishing and latching message. Press ctrl-C to terminate
```

Check the output of the *listener* in console nr. 4

```
[ INFO] [1486054667.604322265]: I heard: [hello world 28202]
[ INFO] [1486054667.704264199]: I heard: [hello world 28203]
[ INFO] [1486054667.804389050]: I heard: [hello world 28204]
[ INFO] [1486054707.646404550]: I heard:
```

Display Own Text

Exercise: Publisher and Subscriber

1. Creating a ROS Package
   - http://wiki.ros.org/ROS/Tutorials/CreatingPackage
2. Writing a Simple Publisher and Subscriber (Python)
   - http://wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber%28python%29

Natural Speech Interaction with ML

Speech Synthesis

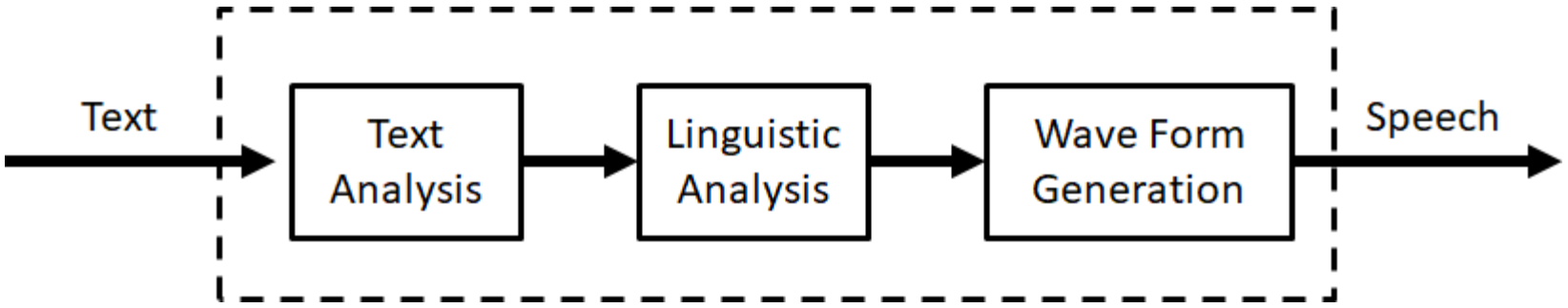
a. Text Analysis (Front-End)
b. Acoustic Modeling
c. Vocoder (Waveform Generation)

Speech Recognition

a. Acoustic Modeling
b. Language Modeling
c. Feature Extraction
d. End-to-End Deep Learning

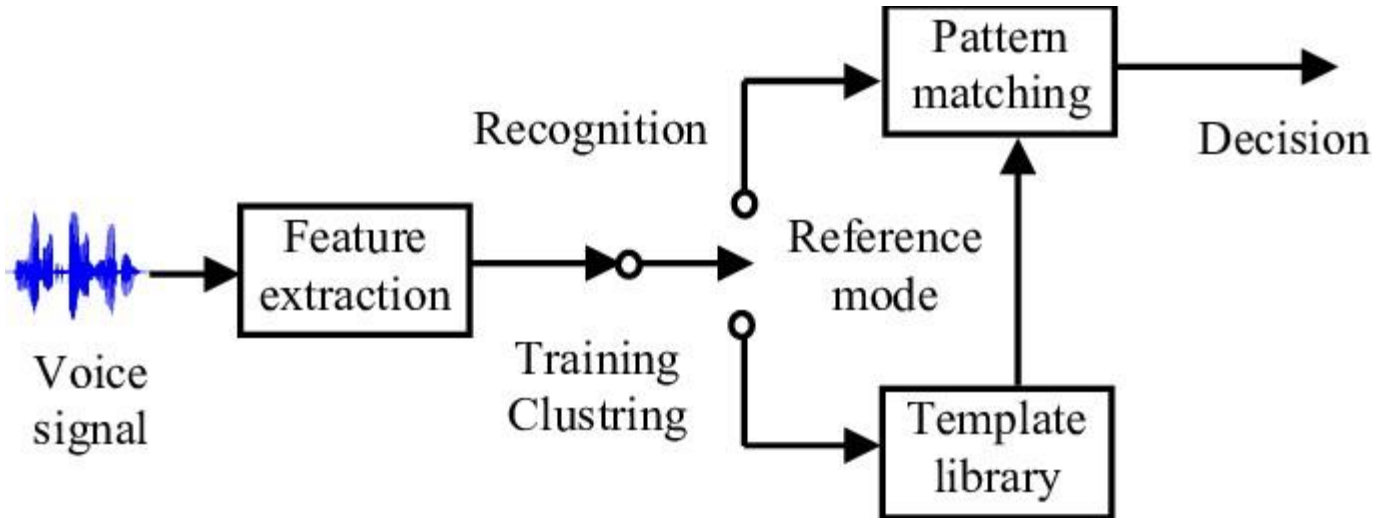
Speech Synthesis (Text-to-Speech)

- Speech synthesis is the production of artificial human speech by a computer or other machines.
- Also known as a text-to-speech (TTS) system, where normal language text is converted into speech.



Speech Recognition

- Speech recognition is a process to extract speech contents from human's voice.
- An automatic speech recognition (ASR) system converts spoken words into text.



[Hamidia, Mahfoud, et al. "Voice interaction using Gaussian mixture models for augmented reality applications." 2015 4th International Conference on Electrical Engineering (ICEE). IEEE, 2015.]

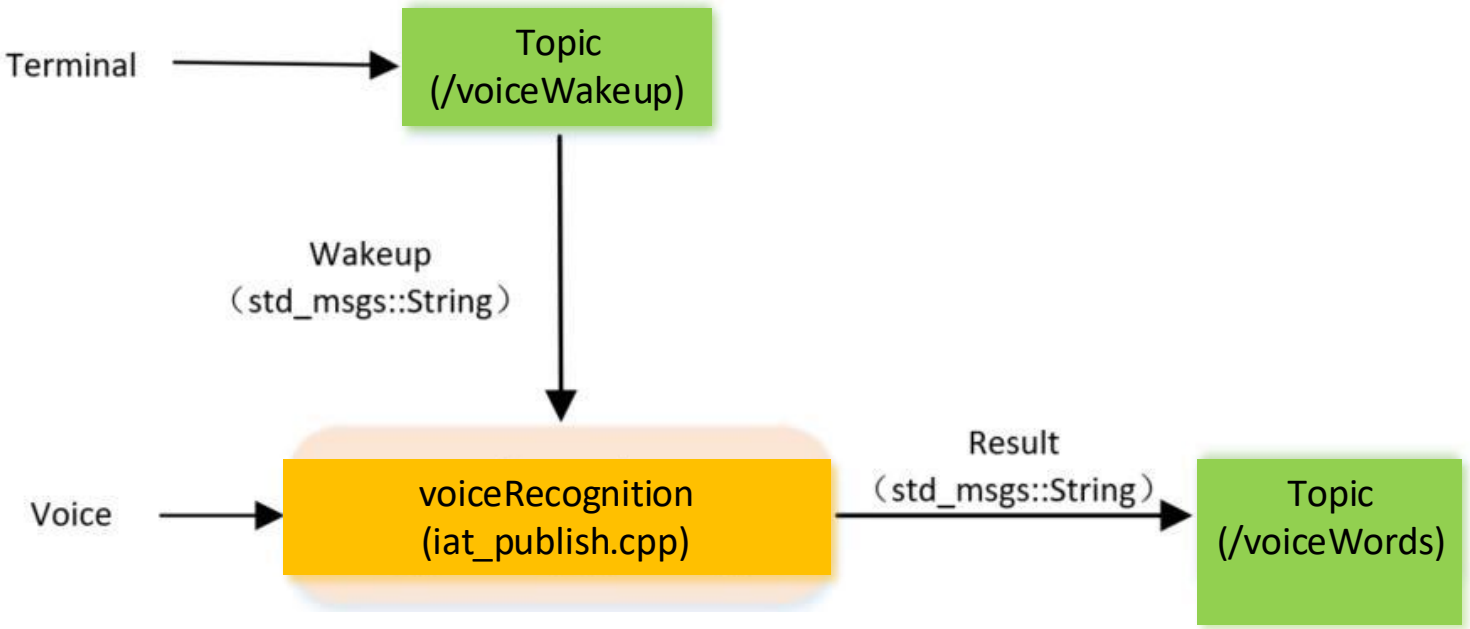
iFlytek Open Platform



https://www.xfyun.cn/

Speech Recognition

- **Speech recognition function block diagram:**



Speech Recognition

- **Speech recognition code:**

  - subscriber: Receives the wakeup signal and sets the wakeupFlag variable;
  - publisher: In the main loop, calls the SDK's speech recognition function, sets the resultFlag variable after successful recognition, and publishes the recognized string.

```
ros::init(argc, argv, "voiceRecognition");
ros::NodeHandle n;
ros::Rate loop_rate(10);

ros::Subscriber wakeUpSub = n.subscribe("voiceWakeup", 1000, WakeUp);
ros::Publisher voiceWordsPub = n.advertise<std_msgs::String>("voiceWords", 1000);

ROS_INFO("Sleeping...");
int count = 0;

int ret = MSP_SUCCESS;
int upload_on = 1; /* whether upload the user word */
/* login params, please do keep the appid correct */
const char* login_params = "appid = 2debcb39, work_dir = .";
int aud_src = 0; /* from mic or file */

/*
* See "iFlytek MSC Reference Manual"
*/
const char* session_begin_params =
    "sub = iat, domain = iat, language = zh_cn, "
    "accent = mandarin, sample_rate = 16000, "
    "result_type = plain, result_encoding = utf8";
```

jupiterobot2_voice_xf/src/iat_publish.cpp

Speech Recognition

- **Speech recognition example:**

  $ roscore

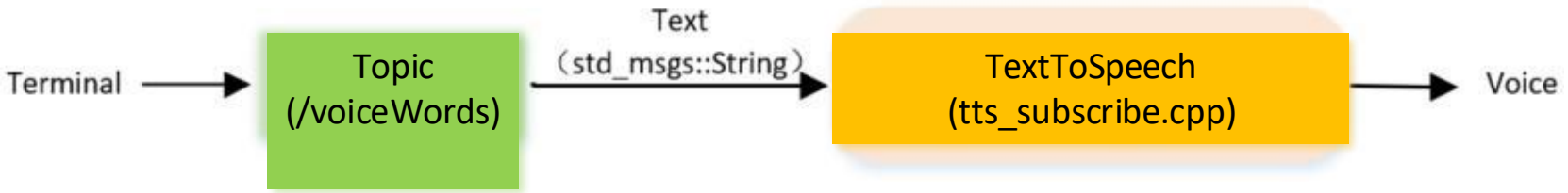
  $ rosrun jupiterobot2_voice_xf iat_publish

  $ rostopic pub /voiceWakeup std_msgs/String "data: 'any string'"



Speech Synthesis

- **Speech synthesis function block diagram:**



Speech Synthesis

- **Speech synthesis code:**

- subscriber: Subscribes to the voiceWords topic and receives input strings.
- voiceWordsCallback: Uses the SDK interface to convert strings into speech.

```
{
  char cmd[2000];
  const char* text;
  int         ret                   = MSP_SUCCESS;
  const char* session_begin_params =
    "voice_name = xiaoyan, text_encoding = utf8, sample_rate = 16000, speed "
    "= 50, volume = 50, pitch = 50, rdn = 2";
  const char* filename              = "tts_sample.wav"; //合成的语音文件名称


  std::cout<<"I heard :"<<msg->data.c_str()<<std::endl;
  text = msg->data.c_str();
  /* 文本合成 */
  printf("开始合成 ...\n");
  ret = text_to_speech(text, filename, session_begin_params);
  if (MSP_SUCCESS != ret)
  {
      printf("text_to_speech failed, error code: %d.\n", ret);
  }
  printf("合成完毕\n");


  unlink("/tmp/cmd");
  mkfifo("/tmp/cmd", 0777);
  popen("mplayer -quiet -slave -input file=/tmp/cmd 'tts_sample.wav'","r");
  sleep(30);
  printf("Mplayer Run Success\n");

}
```

jupiterobot2_voice_xf/src/tts_subscribe.cpp

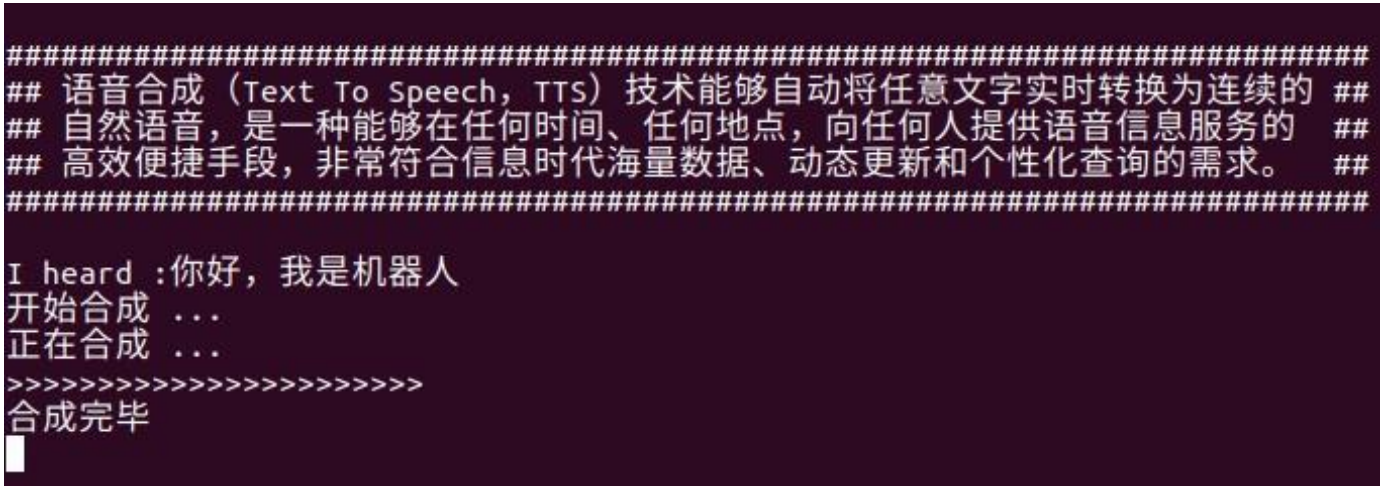
Speech Synthesis

- **Speech synthesis example:**

  $ roscore

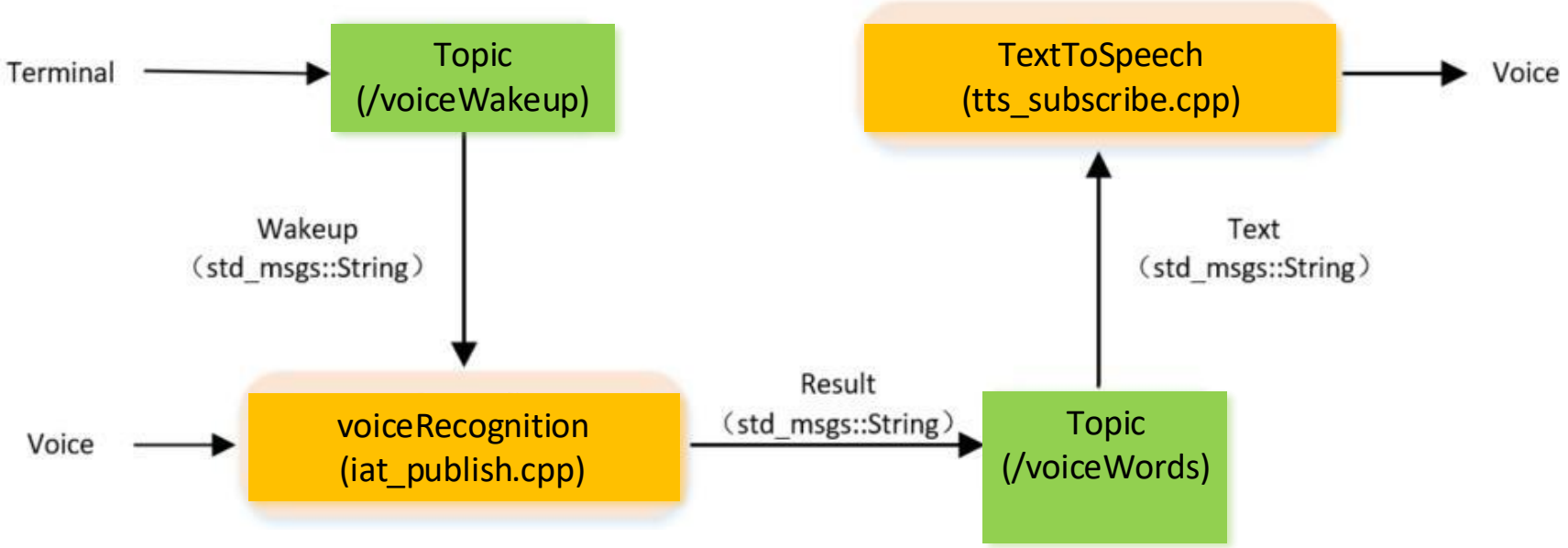
  $ rosrun jupiterobot2_voice_xf tts_subscribe

  $ rostopic pub /voiceWords std_msgs/String "data: '你好，我是机器人'"



Speech Recognition and Speech Synthesis

- **Speech recognition and speech synthesis block diagram:**



Speech Recognition and Speech Synthesis

- **Speech recognition and output example:**

  $ roslaunch jupiterobot2_voice_xf repeat_voice.launch

  $ rostopic pub /voiceWakeup std_msgs/String "data: 'any string'"

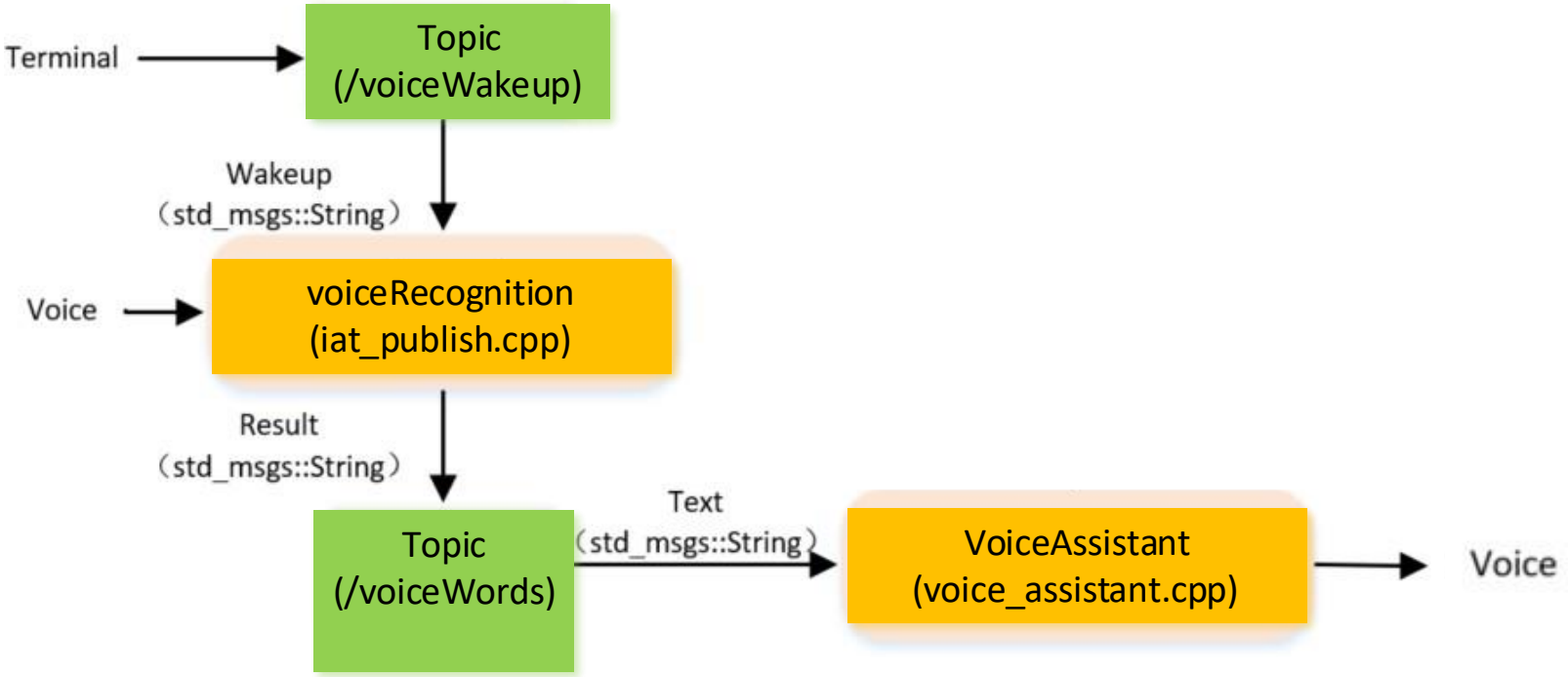
```
################################################################################
## 语音合成（Text To Speech，TTS）技术能够自动将任意文字实时转换为连续的 ##
## 自然语音，是一种能够在任何时间、任何地点，向任何人提供语音信息服务的  ##
## 高效便捷手段，非常符合信息时代海量数据、动态更新和个性化查询的需求。   ##
################################################################################

c:1
c:2
c:3
waking up
c:4
Demo recognizing the speech from microphone
Speak in 15 seconds
Start Listening...
Result:[ 你好我是机器人。 ]

Speaking done
Not started or already stopped.
15 sec passed
c:5
I heard :你好我是机器人。
开始合成 ...
正在合成 ...
c:6
c:7
c:8
>>
合成完毕
c:9
c:10
```

Robot Speech Interaction

- **Speech interaction function block diagram:**



Robot Speech Interaction

- **Speech interaction code:**
  - subscriber: Subscribes to the voiceWords topic and receives input strings.
  - voiceWordsCallback: Uses an if statement to determine keywords and provides a voice response.

```
std::string dataString = msg->data;
if (dataString.find("你是谁") != std::string::npos ||
    dataString.find("名字") != std::string::npos) {
  answer.data = "我是你的语音小助手，你可以叫我小R";
  std::cout << answer.data << std::endl;
} else if (dataString.find("你几岁了") != std::string::npos ||
           dataString.find("年龄") != std::string::npos) {
  answer.data = "我已经四岁了，不再是两三岁的小孩子了";
  std::cout << answer.data << std::endl;
} else if (dataString.find("你可以做什么") != std::string::npos ||
           dataString.find("干什么") != std::string::npos) {
  answer.data = "你可以问我现在时间";
  std::cout << answer.data << std::endl;
} else if (dataString.find("时间") != std::string::npos) {
  // 获取当前时间
  struct tm* ptm;
  time_t ts = time(NULL);
  ptm = localtime(&ts);
  answer.data = "现在时间" + ToString_(ptm->tm_hour) + "点" +
                ToString_(ptm->tm_min) + "分";
  std::cout << answer.data << std::endl;
} else {
  answer.data = msg->data.c_str();
}
```

jupiterobot_voice_xf/src/voice_assistant.cpp

Robot Speech Interaction

- **Speech interaction example:**

  $ roslaunch jupiterobot2_voice_xf voice_assistant.launch

  $ rostopic pub /voiceWakeup std_msgs/String "data: 'any string'"

```
waking up
c:4
Demo recognizing the speech from microphone
Speak in 15 seconds
Start Listening...
Result:[ 你是谁？ ]

Speaking done
Not started or already stopped.
15 sec passed
c:5
I heard :你是谁？
我是你的语音小助手，你可以叫我小R
I heard :我是你的语音小助手，你可以叫我小R
开始合成 ...
正在合成 ...
c:6
c:7
c:8
c:9
c:10
c:11
>>>>
合成完毕
```

```
waking up
c:4
Demo recognizing the speech from microphone
Speak in 15 seconds
Start Listening...
Result:[ 你几岁了？ ]

Speaking done
Not started or already stopped.
15 sec passed
c:5
I heard :你几岁了？
我已经四岁了，不再是两三岁的小孩子了
I heard :我已经四岁了，不再是两三岁的小孩子了
开始合成 ...
正在合成 ...
c:6
c:7
c:8
c:9
c:10
c:11
c:12
>>>>>
合成完毕
```

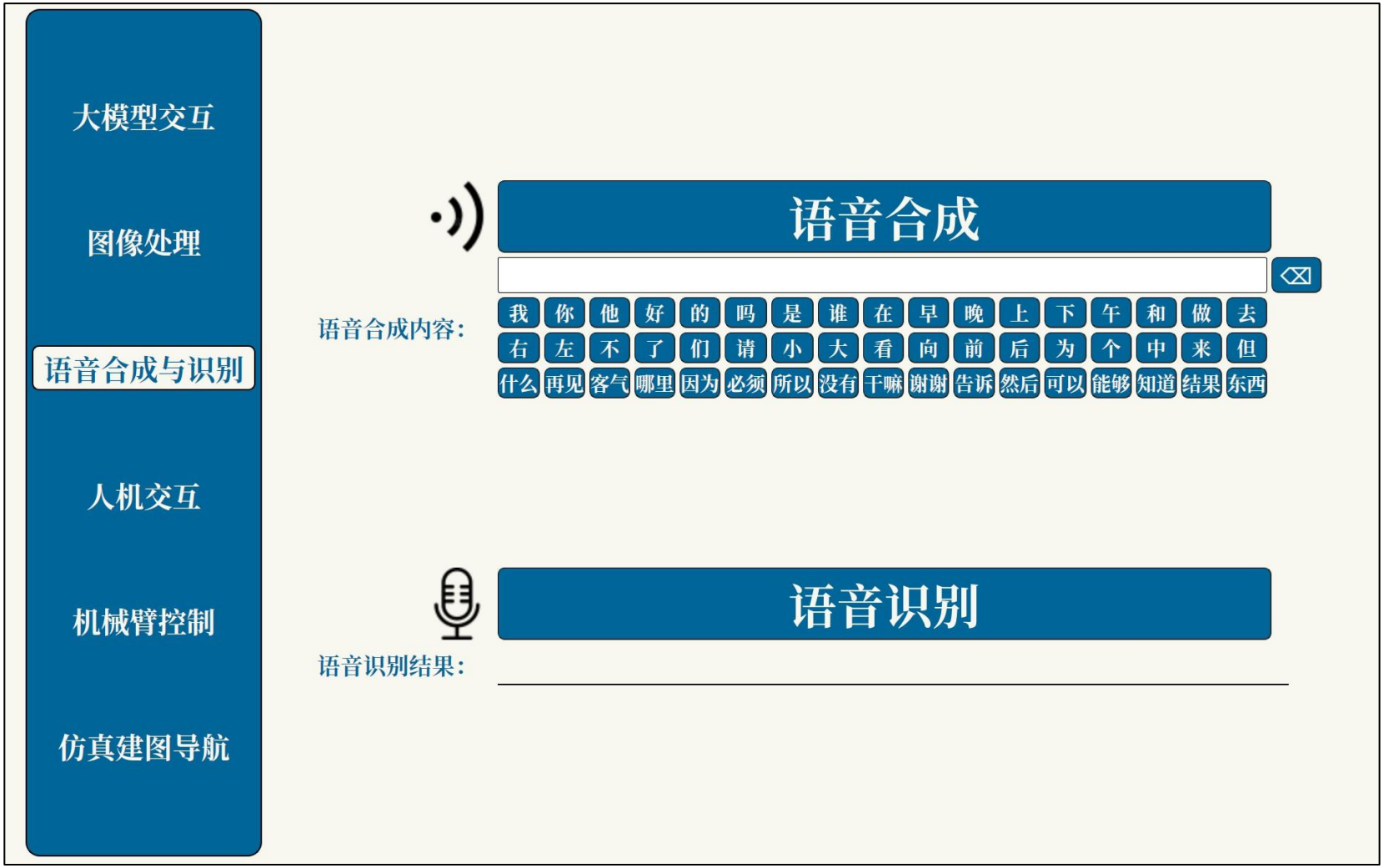
```
waking up
c:4
Demo recognizing the speech from microphone
Speak in 15 seconds
Start Listening...
Result:[ 现在时间。 ]

Speaking done
Not started or already stopped.
15 sec passed
c:5
I heard :现在时间。
现在时间15点22分
I heard :现在时间15点22分
开始合成 ...
正在合成 ...
c:6
c:7
c:8
c:9
>>>
合成完毕
```

Jupiter Robot – Speech Demo GUI

Execution

- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch

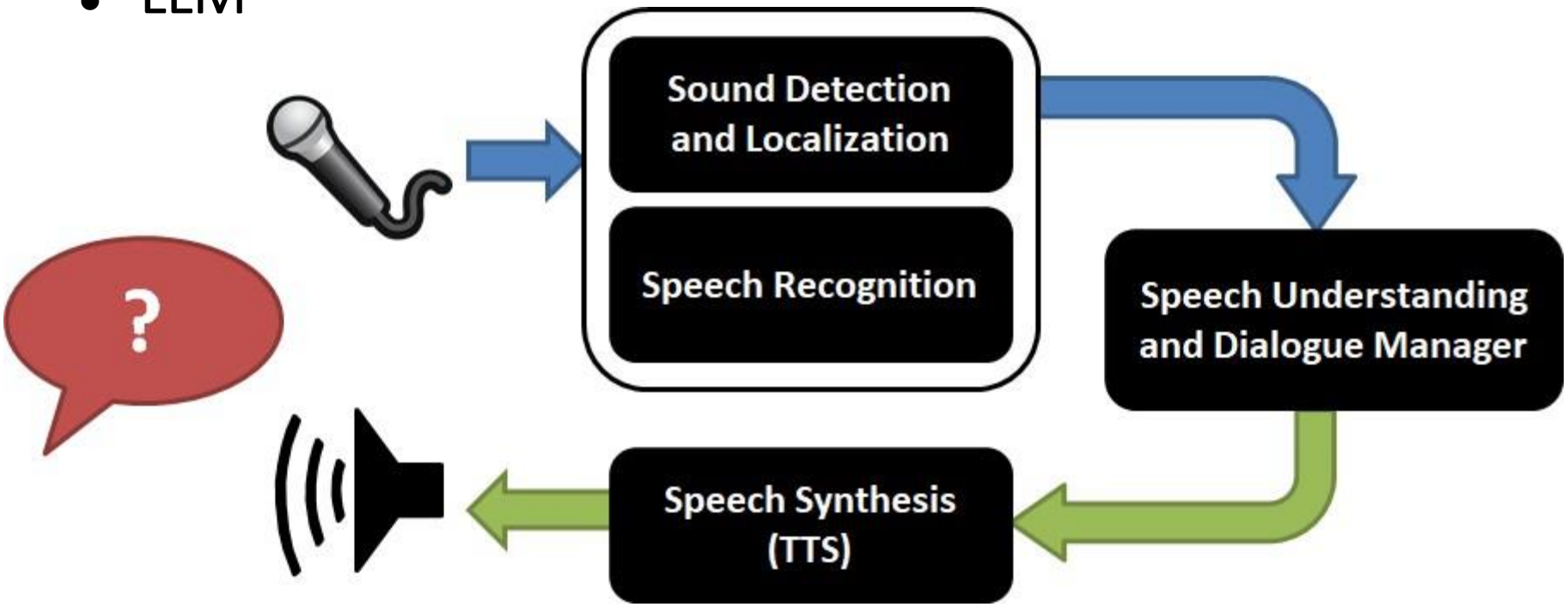




CHATBOT APPLICATION

Chatbot System

- Sound Detection and Localization
- Speech Understanding and Dialogue Manager
- Dialog System
- LLM



Exercise: Chatbot Application

1. Combine the TTS and SR nodes to create a simple Q&A chatbot.

Jupiter Robot – LLM Demo GUI

Execution

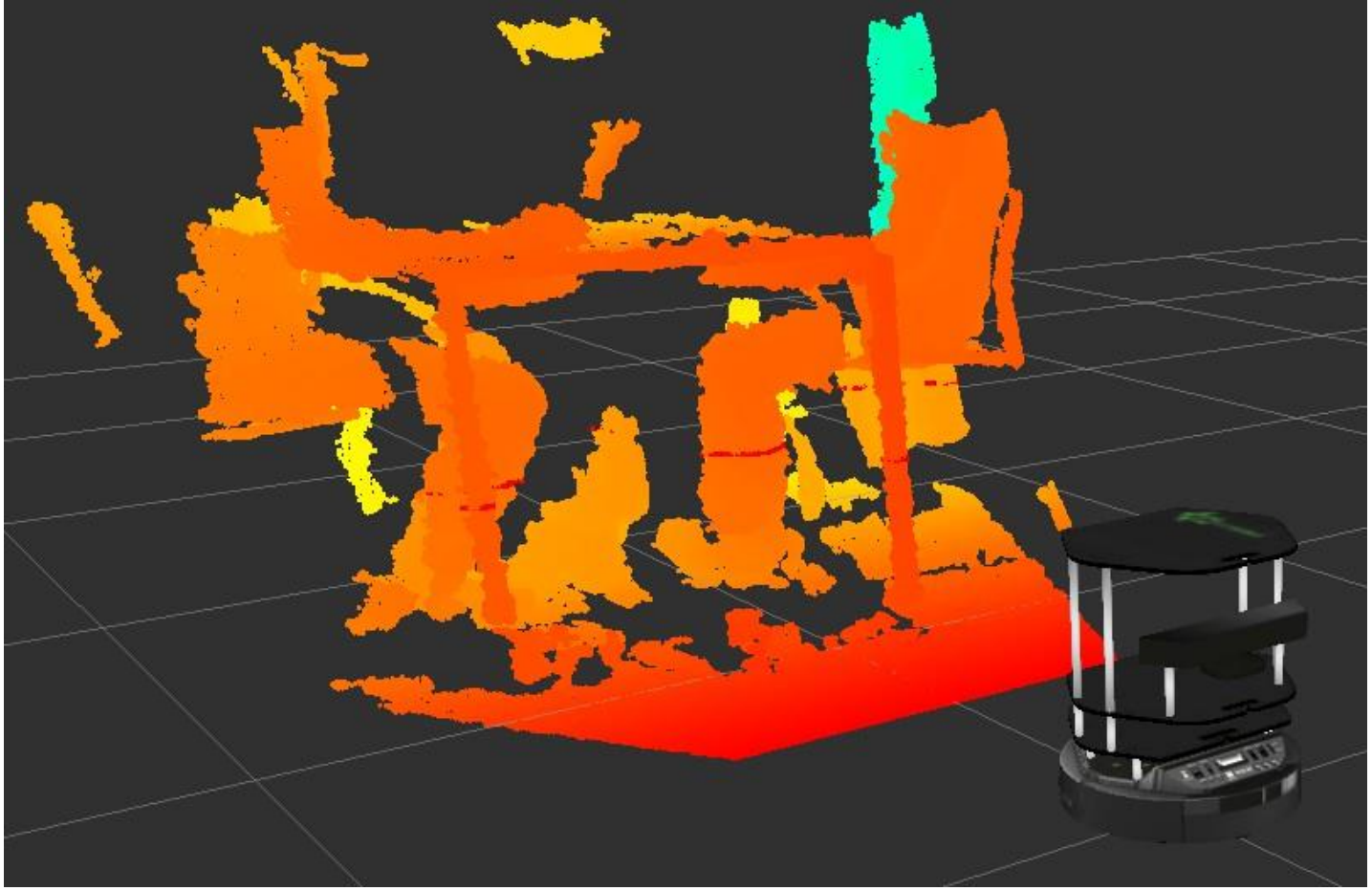
- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch



Chatbot with LLM

- OpenAI API – https://platform.openai.com/docs/overview
- Setup

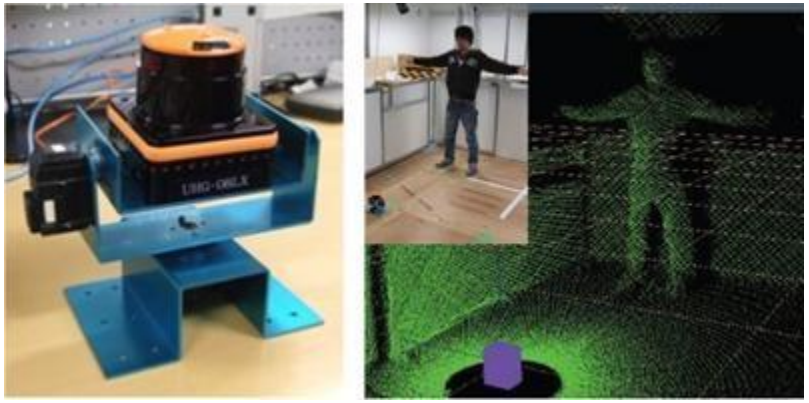
  Git clone RobotForAll Speech sample code into workspace
  - $ git clone https://github.com/robotforall/rfa-speech.git
  - $ pip install openai
  - **deepseek.yaml, qwen.yaml**: OPENAI_API_KEY: "Insert your API Key here"
- Sample code execution
  - $ roslaunch rfa_gpt **deepseek.launch** OR **qwen.launch**
  - $ rostopic pub -1 /gpt_in std_msgs/String “data: ‘hello’”



RGB-D SENSING

Robot Vision Sensors

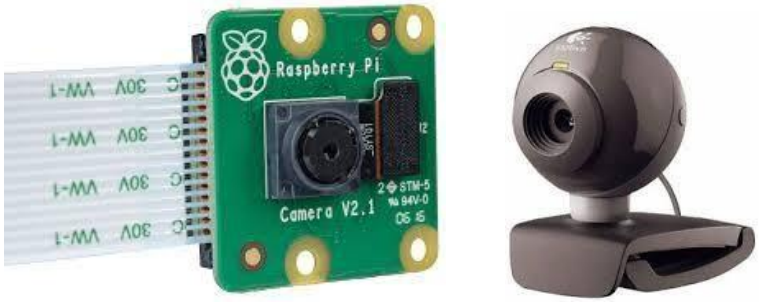
- Optical and camera based systems
- Laser range finder, ultrasonic distance sensor, 3 dimensional range image camera using modulated infrared light, etc.



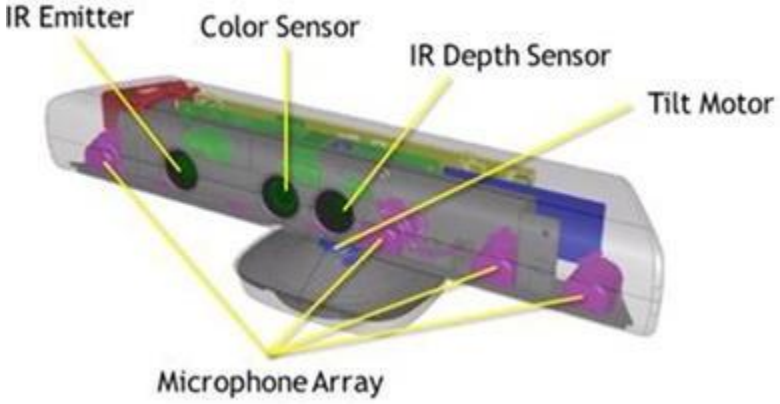
3D laser range finder



2D laser range finder



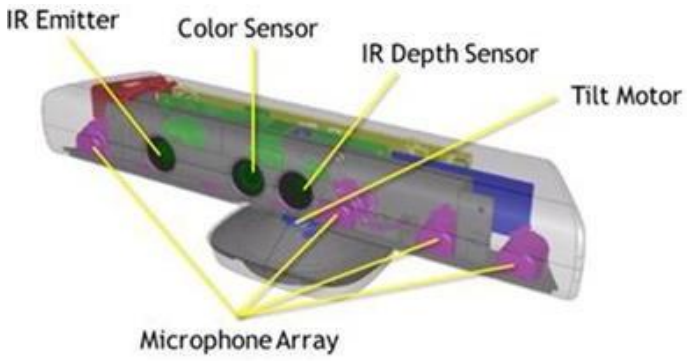
Color camera

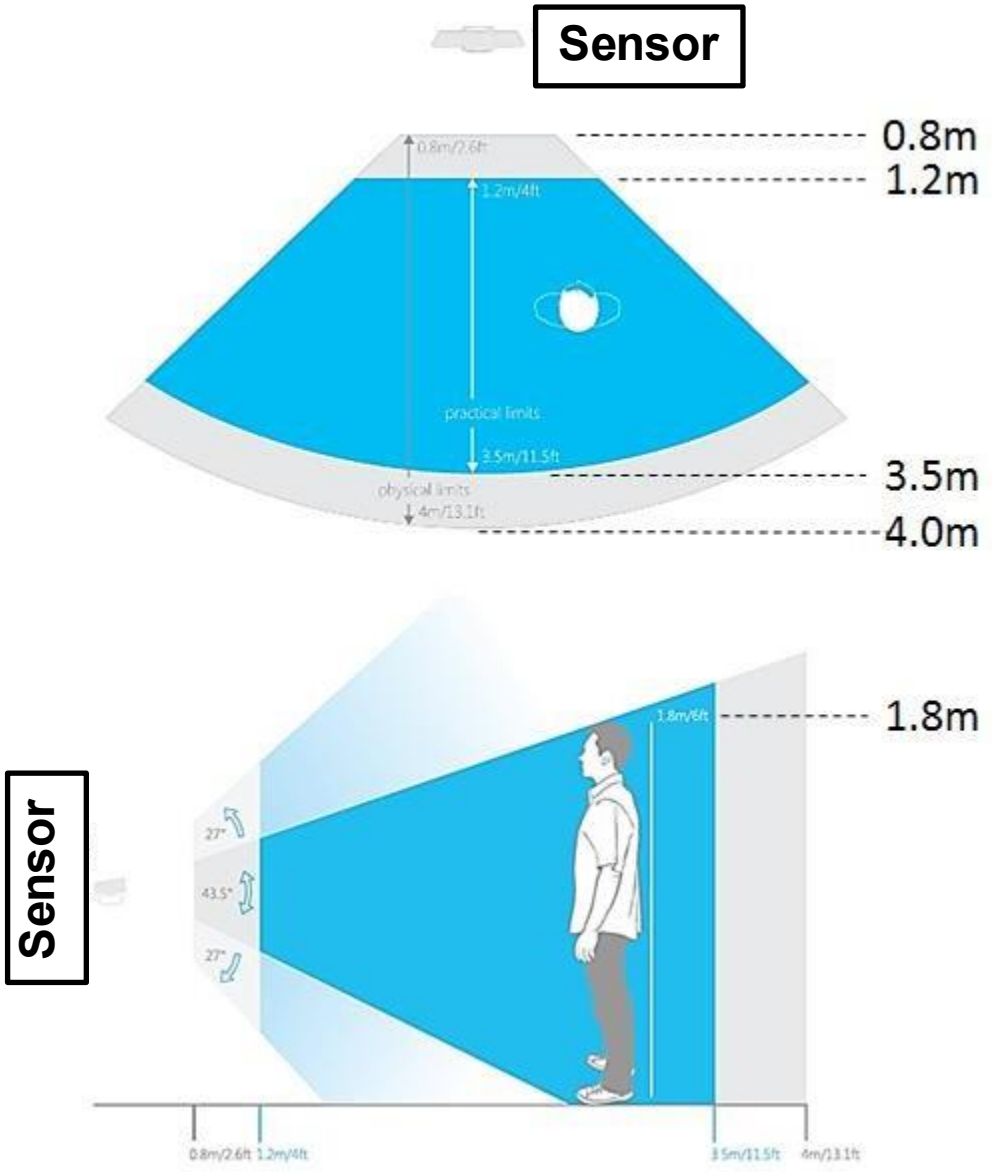


[https://msdn.microsoft.com/en-us/library/jj131033.aspx]

RGB-D Sensor

- 3D MS Kinect sensor
  - RGB Color VGA Video camera
  - Depth sensor
    - Infrared projector
    - Monochrome CMOS sensor
- Resolution 640 x 480 pixel
- 30 FPS

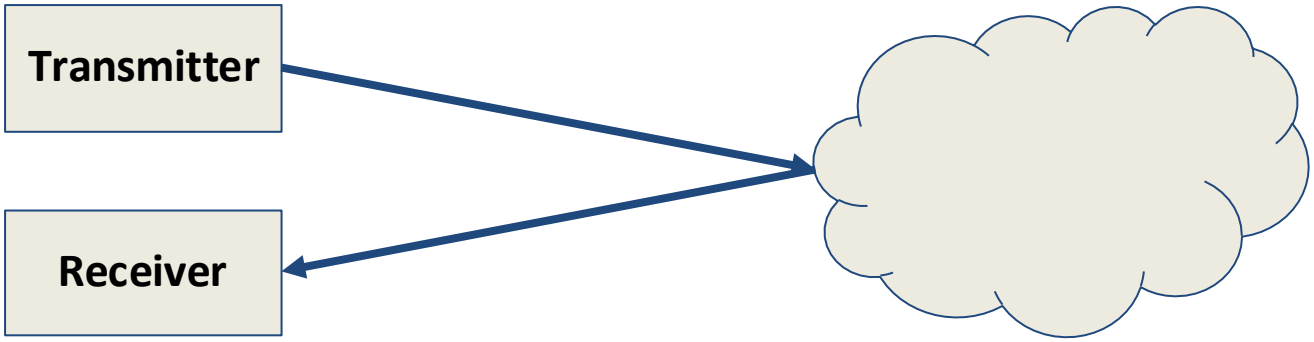




Distance Sensing

The distance to the object is measured using the time (Time of Flight, TOF) of the laser beam emitted from the transmitter hits the object and returns to the receiver.

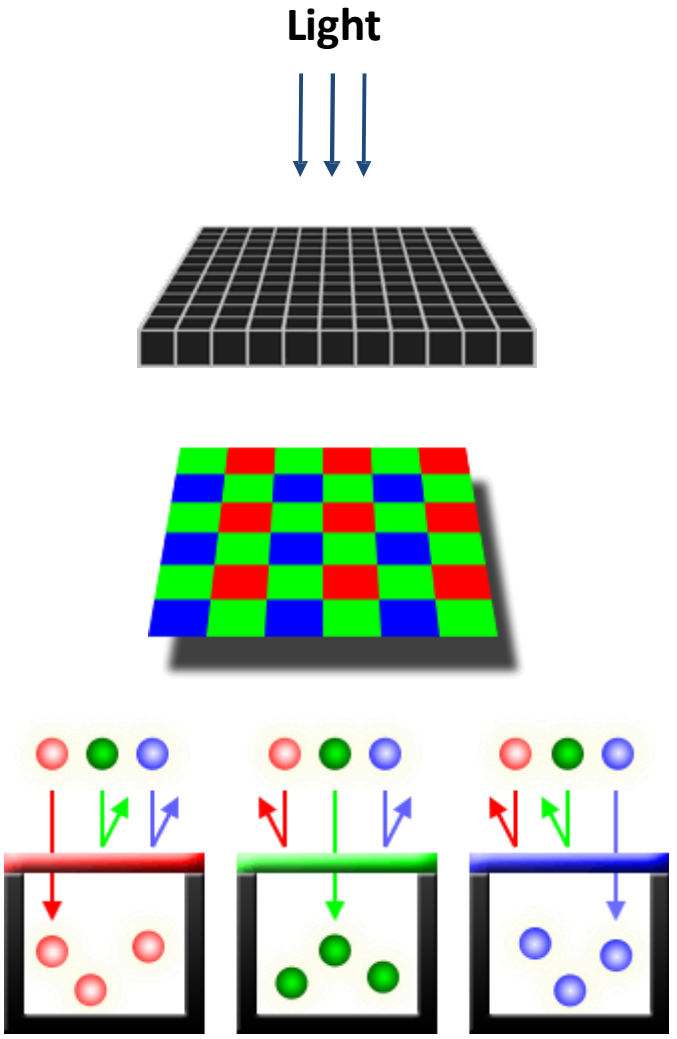
$$l = v\Delta t \,/\, 2$$



Color Sensing

How camera sensor works

- Imaging element
- Light intensity → Electric signal
- CCD(Charge Coupled Device)
- CMOS(Complementary Metal Oxide Semiconductor)
- Photodiode
- Color filter: R(red), G(green), B(blue)
- Pixel value: 256 levels (0 – 255)



[https://www.cambridgeincolour.com/tutorials/camera-sensors.htm]

Visual Perception

- OpenNI2 - http://structure.io/openni
- OpenKinect - https://github.com/OpenKinect
- PCL - http://pointclouds.org/
- OpenCV - http://opencv.org/
- ROS opencv_apps - http://wiki.ros.org/opencv_apps
- Bring up
  - [Jupiter2] $ roslaunch jupiterobot2_bringup **multi_camera.launch**
  - [Juno2] $ roslaunch astra_camera **astra.launch**
- Display RGB and Depth images
  - $ rqt
  - $ rosrun image_view image_view image:=/camera/color/image_raw
  - $ rosrun image_view image_view image:=/camera/depth/image_raw

  * [Jupiter2 bottom camera] Replace “camera” with “camera_bot”

Take Photo

Bring up camera

- [Jupiter2] $ roslaunch jupiterobot2_bringup **multi_camera.launch**
- [Juno2] $ roslaunch astra_camera **astra.launch**

Take photo

- $ rosrun rchomeedu_vision **take_photo.py**

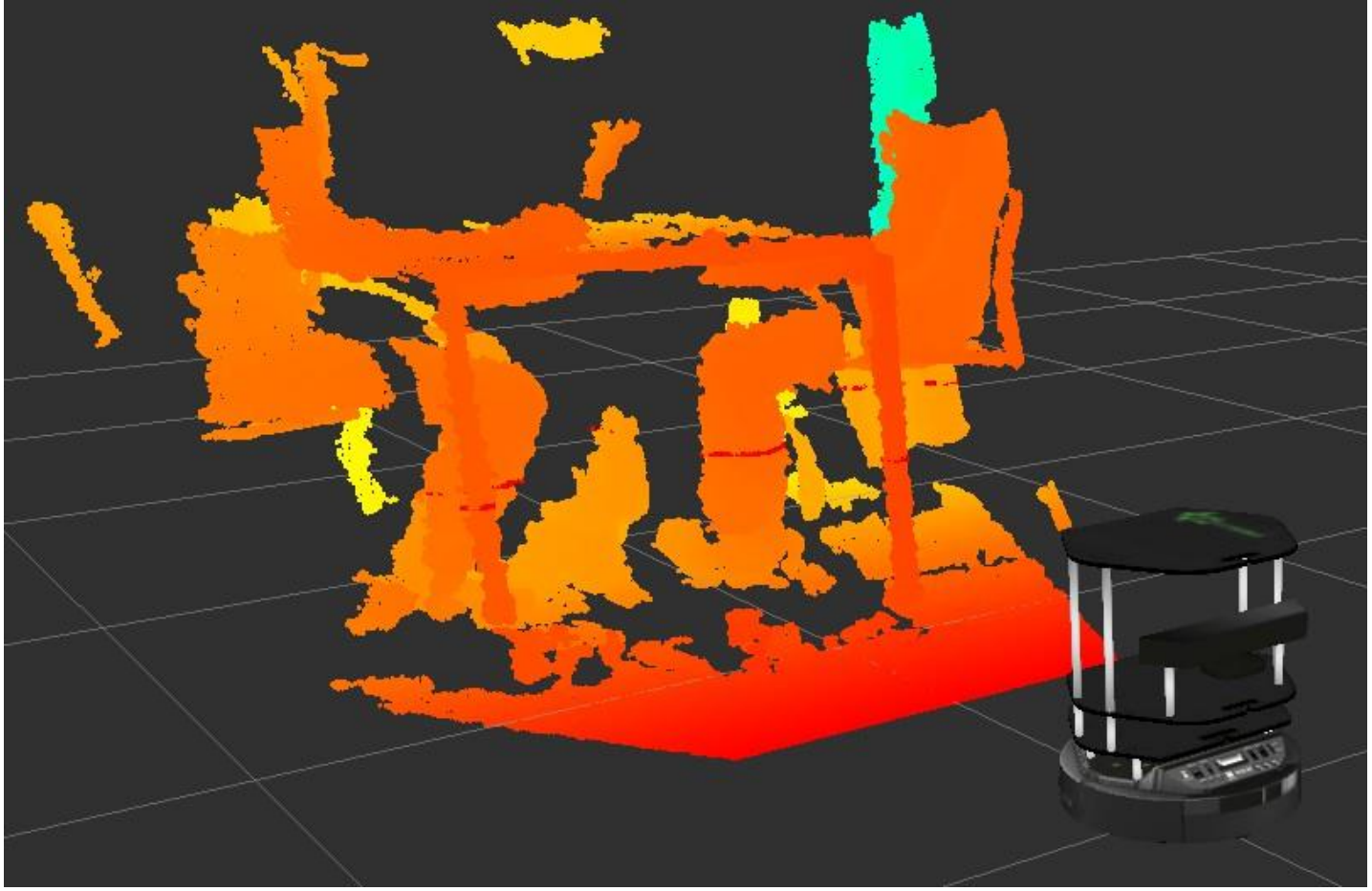
OR

- $ rosrun rchomeedu_vision **take_photo_sub.py**
- $ rostopic pub -1 /take_photo std_msgs/String “take photo”

* Image topic = “/camera/color/image_raw”
  (edit code according to the camera used)
* Photo taken is saved in the current folder

Exercise: Take Photo by Voice Command

1. Combine the Take Photo example with speech module to create a photo taking application by voice command.



OPENCV APPLICATIONS

CamShift Object Tracking

Bring up

- [Jupiter2] $ roslaunch jupiterobot2_bringup **multi_camera.launch**
- [Juno2] $ roslaunch astra_camera **astra.launch**

Image topic argument

- image:=/camera/color/image_raw

OpenCV CamShift

- $ roslaunch opencv_apps **camshift.launch** image:=/camera/color/image_raw

* Select an area of similar color to track

Face/People Detection and Recognition

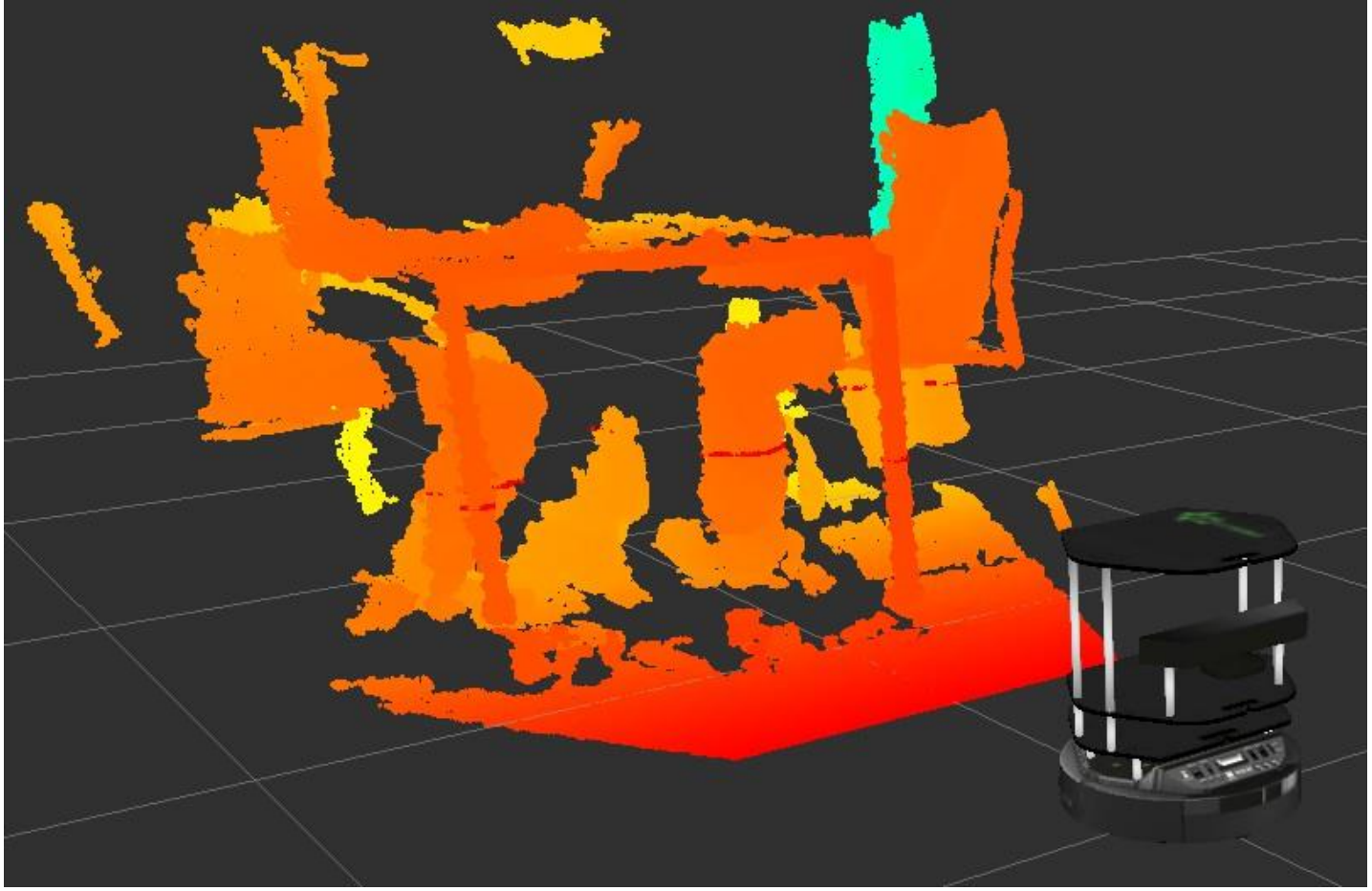
- Bring up
  - [Jupiter2] $ roslaunch jupiterobot2_bringup **multi_camera.launch**
  - [Juno2] $ roslaunch astra_camera **astra.launch**

- Image topic argument
  - image:=/camera/color/image_raw

- Face Detection using Haar feature-based cascade classifiers
  - $ roslaunch opencv_apps **face_detection.launch <**img topic arg>

- Face Recognition
  - $ roslaunch opencv_apps **face_recognition.launch <**img topic arg>

- People Detection using Histogram of Oriented Gradients (HOG)
  - $ roslaunch opencv_apps **people_detect.launch <**img topic arg>



DEEP LEARNING BASED IMAGE PROCESSING

Deep Learning based Image Processing

- Bring up
  - [Jupiter2] $ roslaunch jupiterobot2_bringup **multi_camera.launch**
  - [Juno2] $ roslaunch astra_camera **astra.launch**
- Yolo Object Detection
  - $ roslaunch jupiterobot2_vision_yolov5 **object_detection.launch**
- Body Pose Landmark Detection
  - $ roslaunch jupiterobot2_vision_mediapipe **mediapipe_pose.launch**
- Face Detection
  - $ roslaunch jupiterobot2_vision_mediapipe **mediapipe_faceDetect.launch**
- Face Landmark Detection
  - $ roslaunch jupiterobot2_vision_mediapipe **mediapipe_faceMesh.launch**
- Hand Landmark Detection
  - $ roslaunch jupiterobot2_vision_mediapipe **mediapipe_hand.launch**

Computer Vision Demo GUI

Execution

- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch



Exercise: Say Hello

1. Extend the Face Detection/Recognition example for the system to say greetings when someone comes in front of the system.

Visual Interaction Demo GUI

Execution

- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch



AI based Vision (VLM)

- OpenAI API – https://platform.openai.com/docs/guides/images-vision
- Setup

  Git clone RobotForAll Vision sample code into workspace
  - $ git clone https://github.com/robotforall/rfa-vision.git
  - **api_key** = "Insert your API Key here“,
  - $ pip install openai
- Sample code execution

  Bring up camera
  - [Jupiter2] $ roslaunch jupiterobot2_bringup multi_camera.launch
  - [Juno2] $ roslaunch astra_camera astra.launch

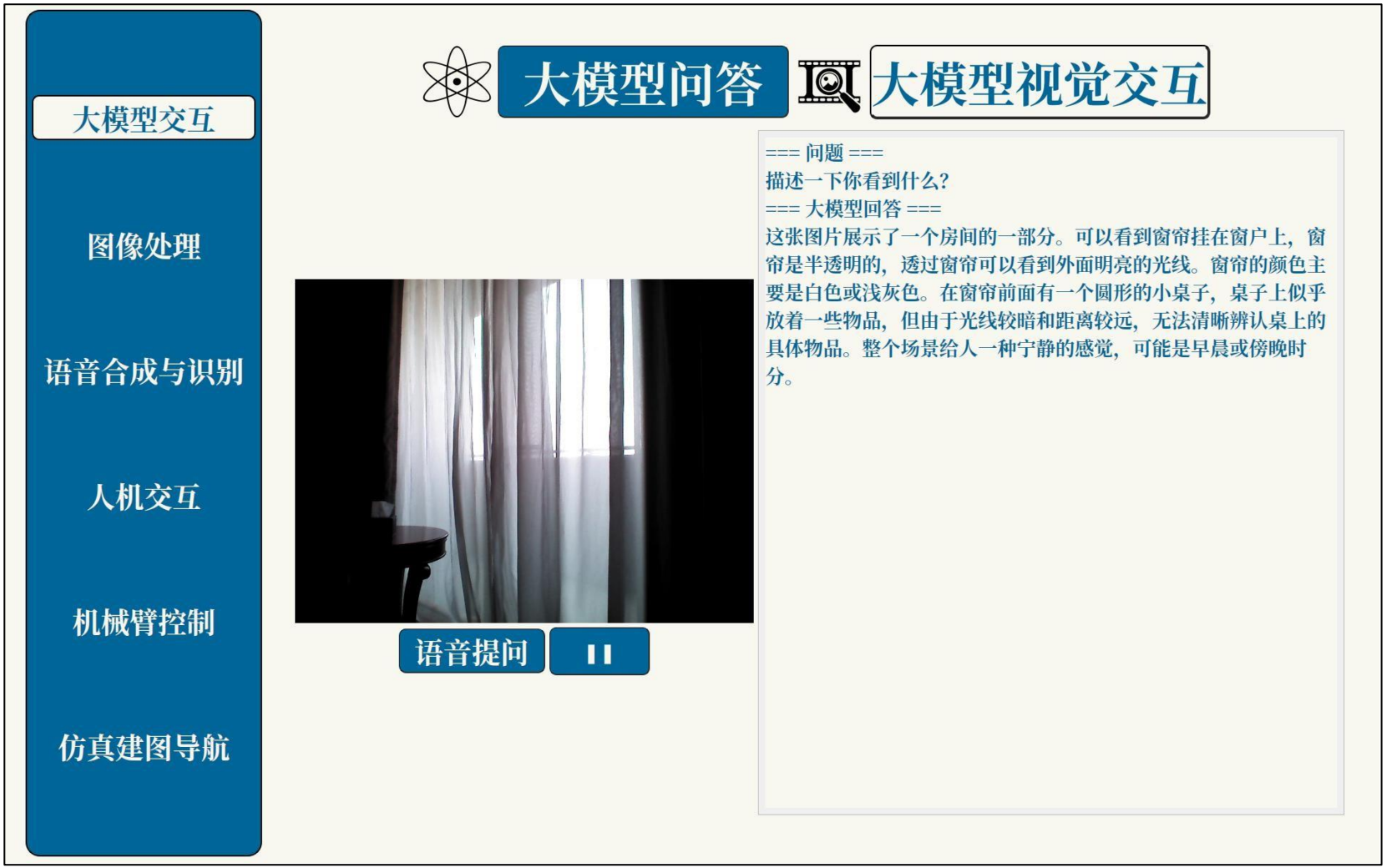
  Image Recognition
  - $ rosrun rfa_vlm **image_recognition_qwen.py**

  Object Detection
  - $ rosrun rfa_vlm **object_detection_qwen.py**

Jupiter Robot – VLM Demo GUI

Execution

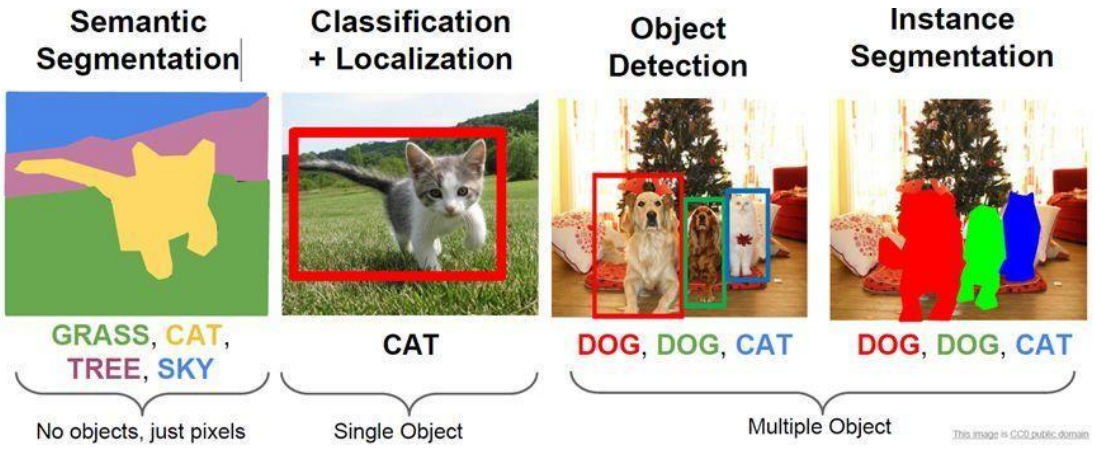
- $ roslaunch jupiterobot2_qt jupiterobot2_qt.launch



Computer Vision with ML

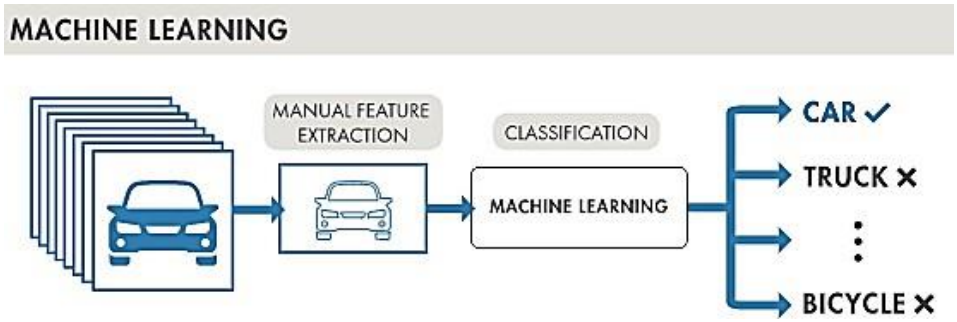
Image Classification

- Object Detection
- Object Classification
- Object Localization
- Image Segmentation



Object Detection: Machine Learning vs. Deep Learning

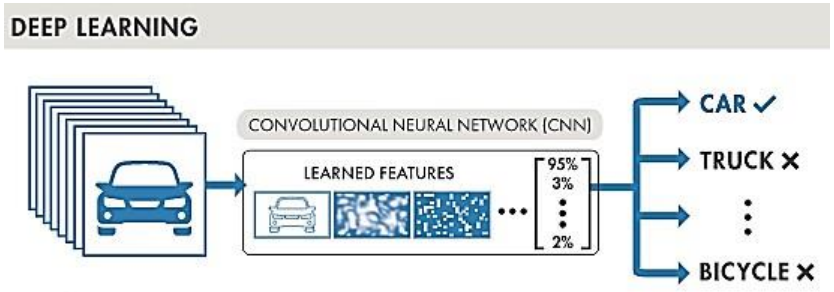
Object detection based on machine learning



- Typically, traditional machine learning algorithms such as Support Vector Machines, AdaBoost, and Random Forests are used.
- The basic concept of these methods is to extract image features, classify those features using a classifier, and then use an object detector to detect objects.
- These algorithms require manual selection and extraction of image features, thus necessitating the knowledge and experience of domain experts.

Object Detection: Machine Learning vs. Deep Learning

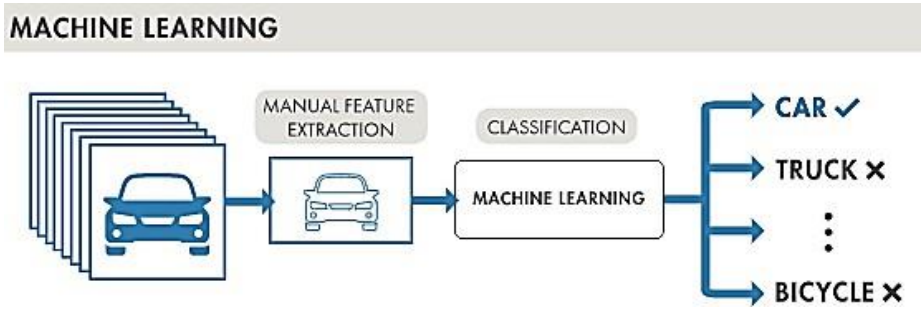
Object detection based on deep learning



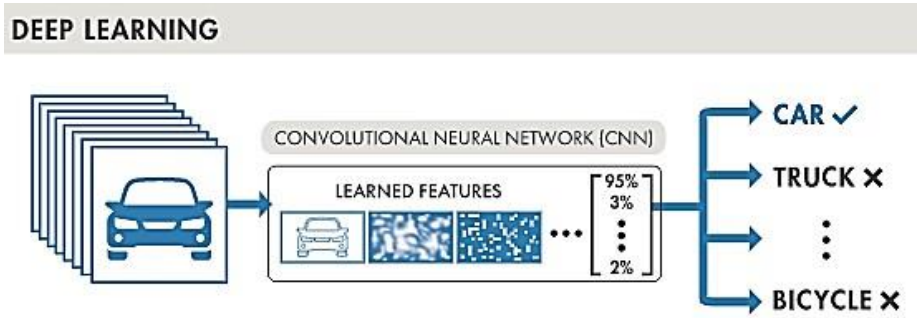
- Deep neural networks are typically used to automatically learn features and perform object detection.
- Currently popular methods fall into two categories: Two-stage detection methods and single-stage detection methods.

Object Detection: Machine Learning vs. Deep Learning

- Machine learning-based object detection

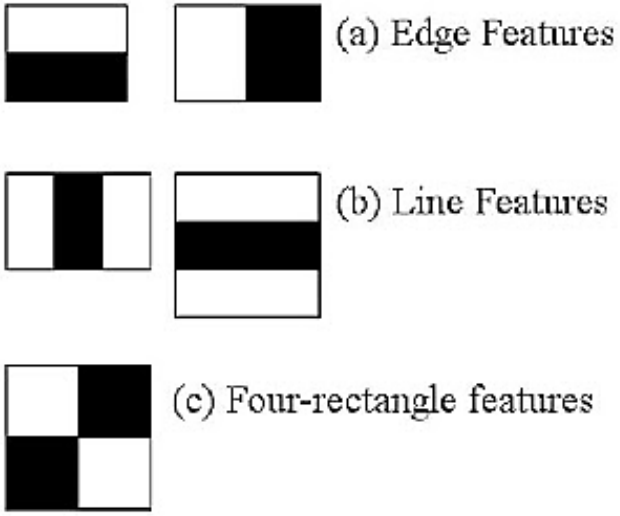


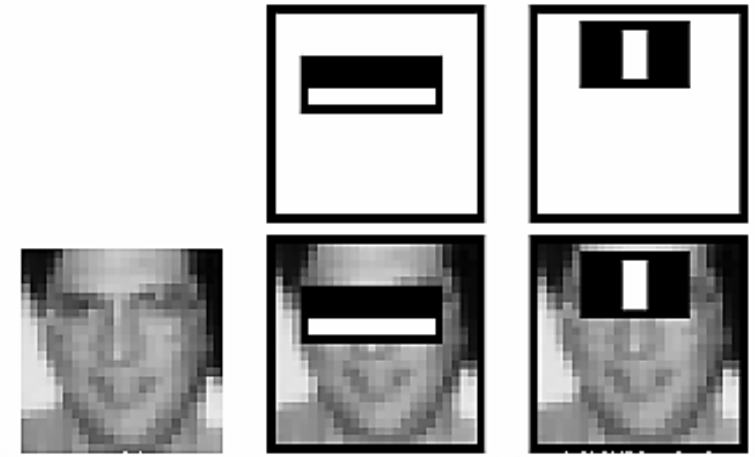
- Deep learning-based object detection



Face Detection based on Machine Learning

- Haar Cascade is a machine learning-based algorithm used to detect objects in images or videos.
- The algorithm uses a set of Haar-like features (rectangular patterns of pixel values) to distinguish objects from the background.
- The algorithm uses these features to train a classifier, which can then be used to detect objects in new images or videos.





Face Detection System

Launch camera

- $ roslaunch astra_camera astra.launch

Launch Face Detection system

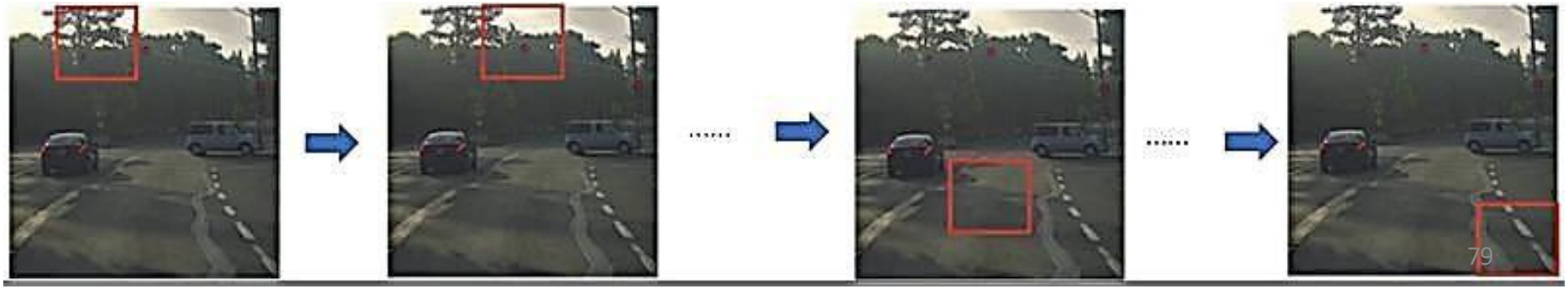
- $ roslaunch opencv_apps face_detection.launch image:=/camera/color/image_raw

Code: https://github.com/ros-perception/opencv_apps



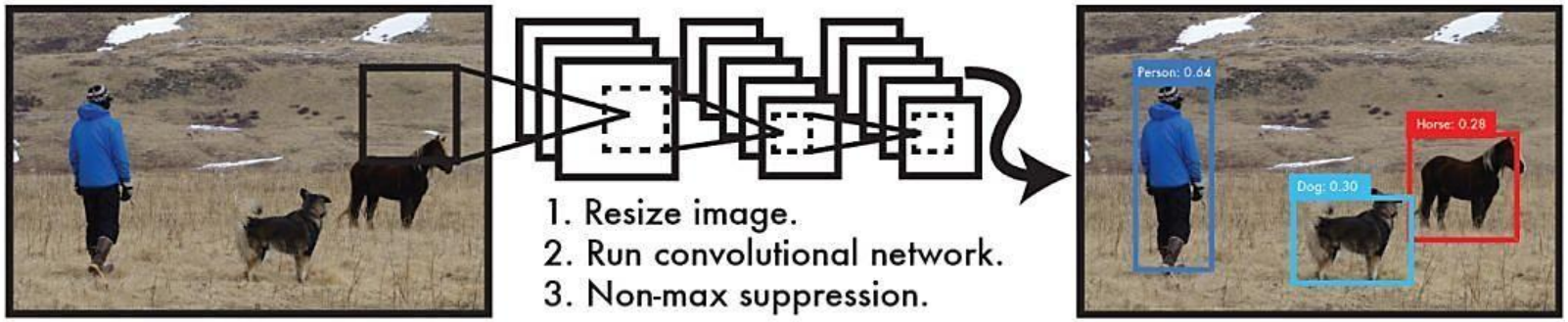
Two-Stage Detection Methods: DPM, R-CNN

- Two-stage process: first, candidate regions are extracted, then classification and regression are performed.
- Sliding window technique
- DPM (Deformable Parts Model)
- R-CNN (Region-based Convolutional Neural Network)

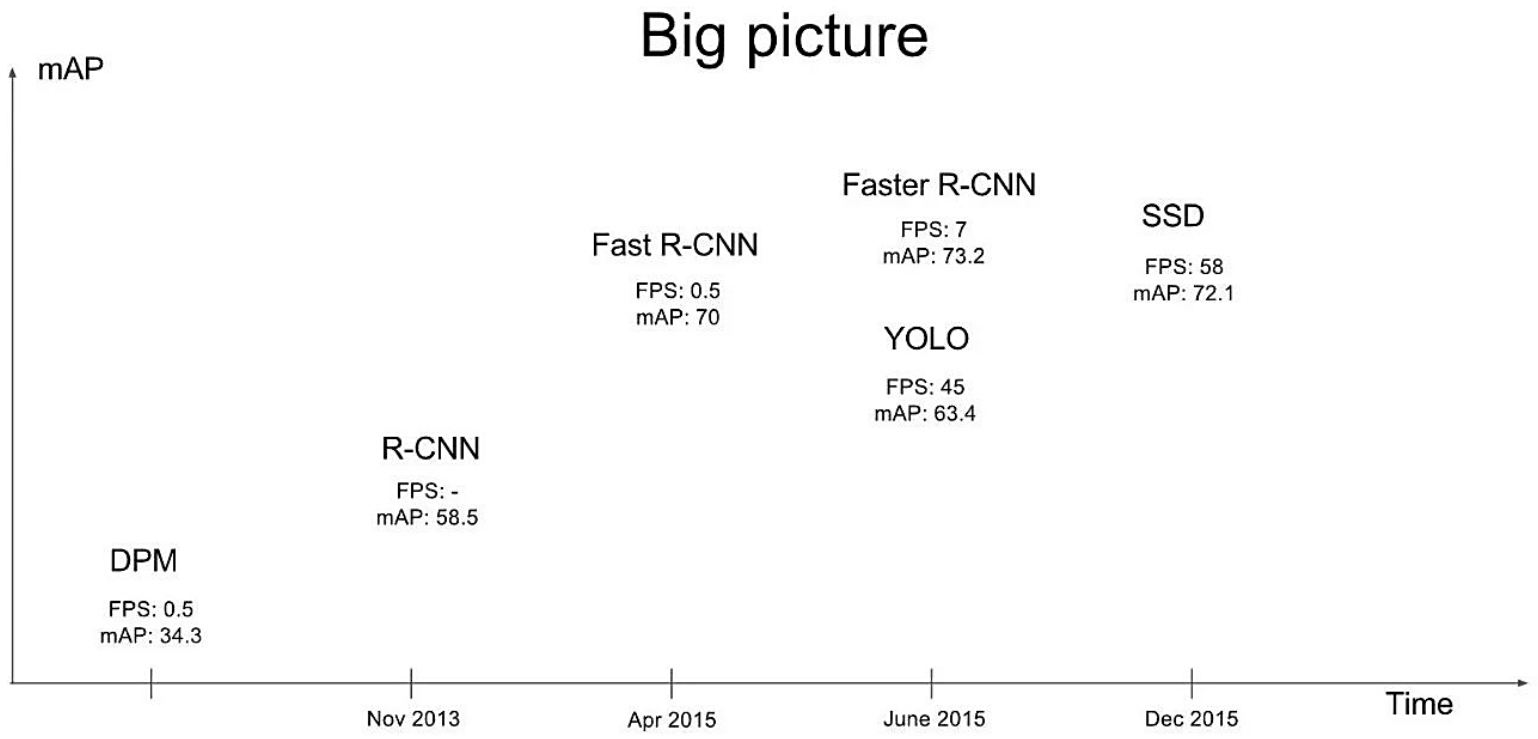


One-Stage Detection Methods: SSD, YOLO

- Single-stage, direct prediction of bounding boxes and categories.
- YOLO (You Only Look Once) is a real-time object detection algorithm.
- Proposed by Joseph Redmon et al. in 2015.
- The YOLO algorithm uses a single CNN model to achieve end-to-end object detection.
- The input image is fed into the CNN network, and the network's predictions are processed to obtain the detected objects.



Advances and Comparisons of Object Detection Algorithms

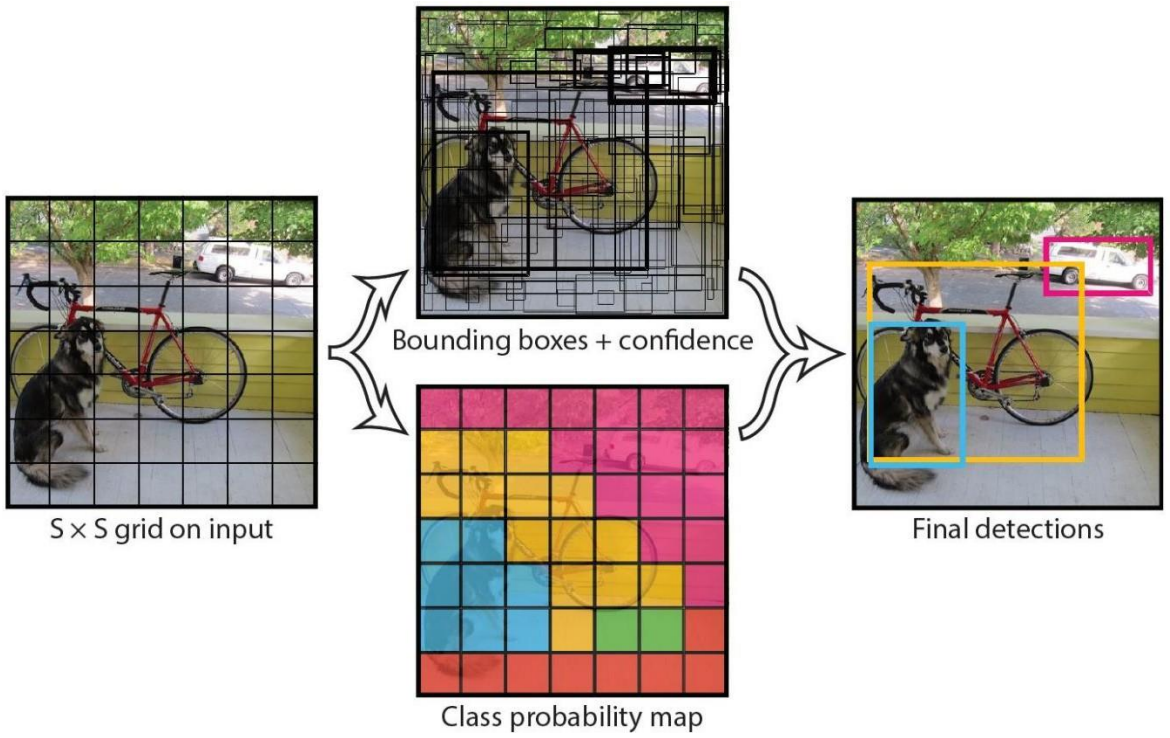


Basic Principles of YOLO

- Step 1: Image Segmentation
  - Divide the input image into an S × S grid.
- Step 2: Bounding Box Prediction
  - Predict B bounding boxes (center coordinates, width, height, and confidence score) for each grid cell.
- Step 3: Class Prediction
  - Predict the probability of C classes for each grid cell.
- Step 4: Non-Maximum Suppression (NMS)
  - Remove overlapping bounding boxes and retain the most likely detections.

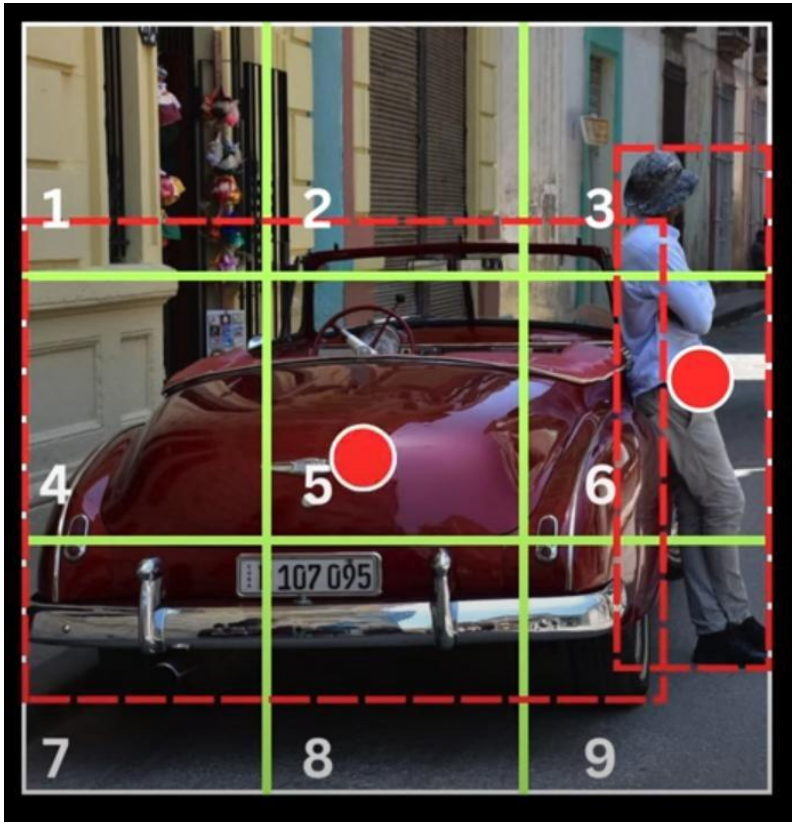
Basic Principles of YOLO

- The image is divided into an S×S grid.
- Each grid cell is responsible for predicting B bounding boxes and class probabilities.
- Redundant boxes are removed using non-maximum suppression (NMS).



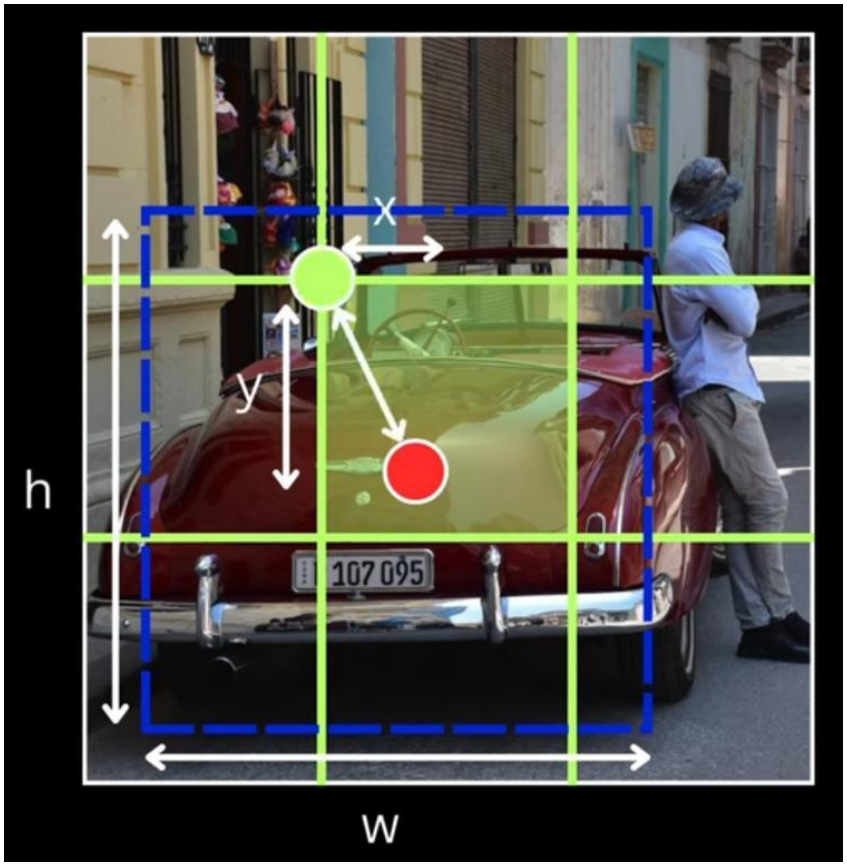
YOLO Image Segmentation

- The input image is segmented into an S*S grid, and each cell is responsible for detecting targets whose center point falls within that cell.
- As you can see, the center point of the car target falls within the middle cell, so that cell is responsible for predicting the car.
- Each cell predicts B bounding boxes and their confidence scores.



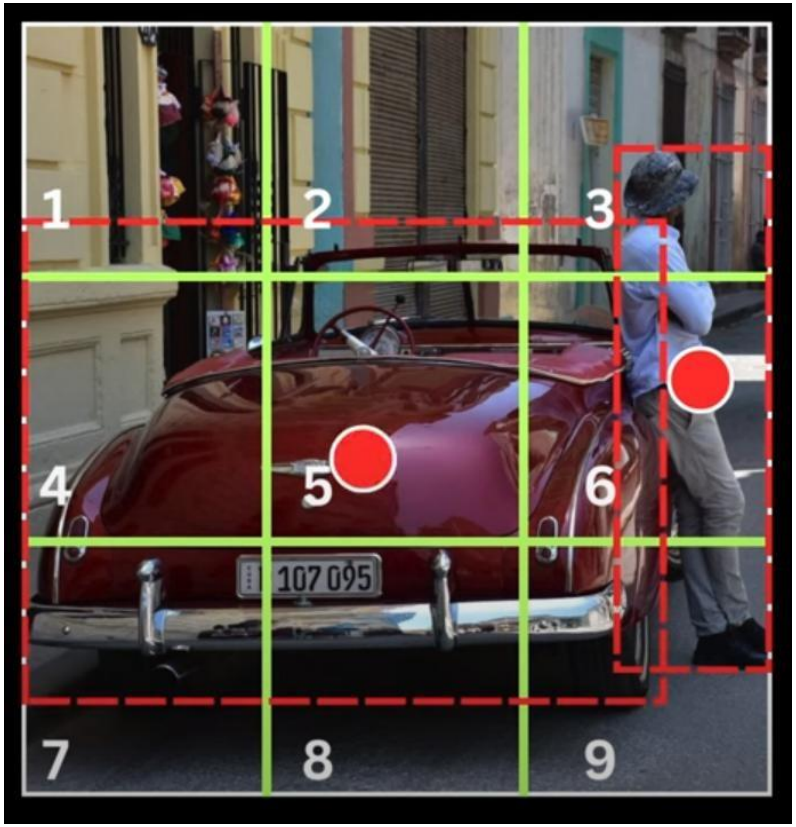
YOLO Bounding Box Prediction

- The size and position of the bounding box can be represented by four values: (x, y, h, w).
  - x and y are the center coordinates of the bounding box. These are offsets relative to the top-left corner of each cell, and the units are relative to the cell size.
  - w and h are the width and height of the bounding box. These are the proportions of the width and height of the bounding box relative to the entire image.
  - Theoretically, the size of these four elements should be within the range [0, 1].



YOLO Bounding Box Prediction

- Each bounding box prediction actually contains 5 elements: (x, y, w, h, c), where the first four represent the size and position of the bounding box, and the last value is the confidence score.
- The confidence score can be defined as Pr(object) * IOU.
  - The probability of containing the object:
    - No object: Pr(object) = 0;
    - Object: Pr(object) = 1.
  - The accuracy of the bounding box is characterized by the IOU (intersection over union) between the predicted box and the ground truth box, denoted as IOU.



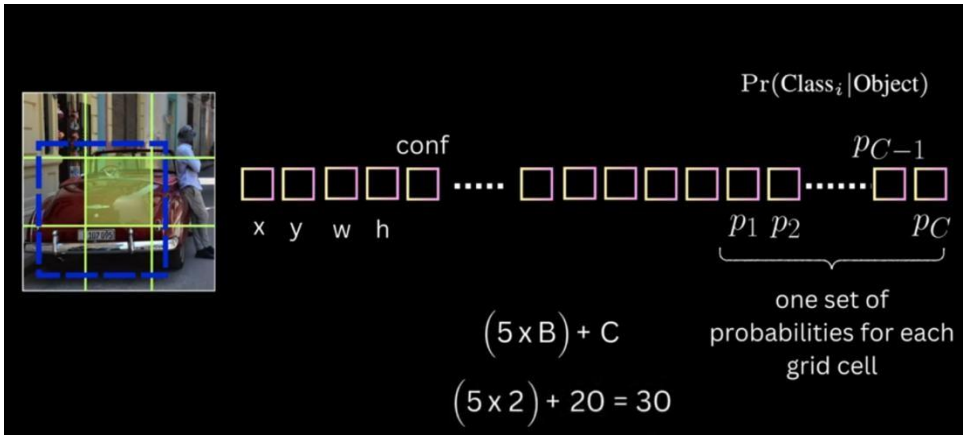
YOLO Category Prediction

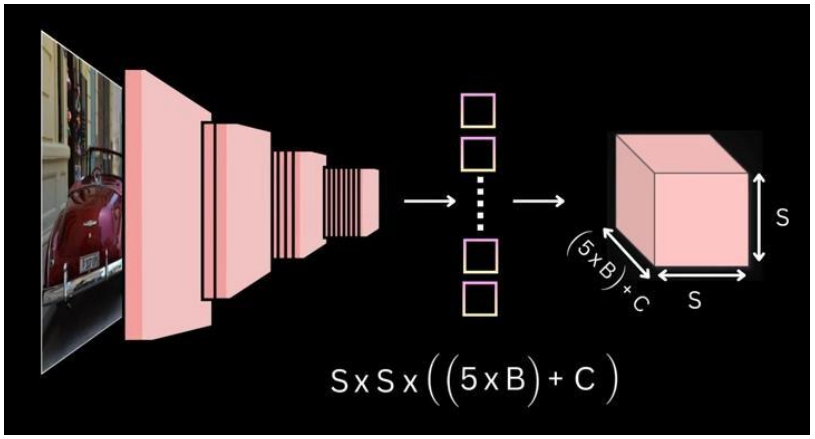
- For each cell, it also provides C predicted class probability values, representing the probability that the bounding box predicted by that cell belongs to each class.
- Simply put, each cell corresponds to only one class with the highest probability. However, these probability values are actually conditional probabilities under the confidence scores of each bounding box, i.e., Pr(classi|object).
- At the same time, we can calculate the class-specific confidence scores for each bounding box:

$$Pr(class_i|object) * Pr(object) * \text{IOU}_{pred}^{truth} = Pr(class_i) * \text{IOU}_{pred}^{truth}$$

YOLO Final Prediction

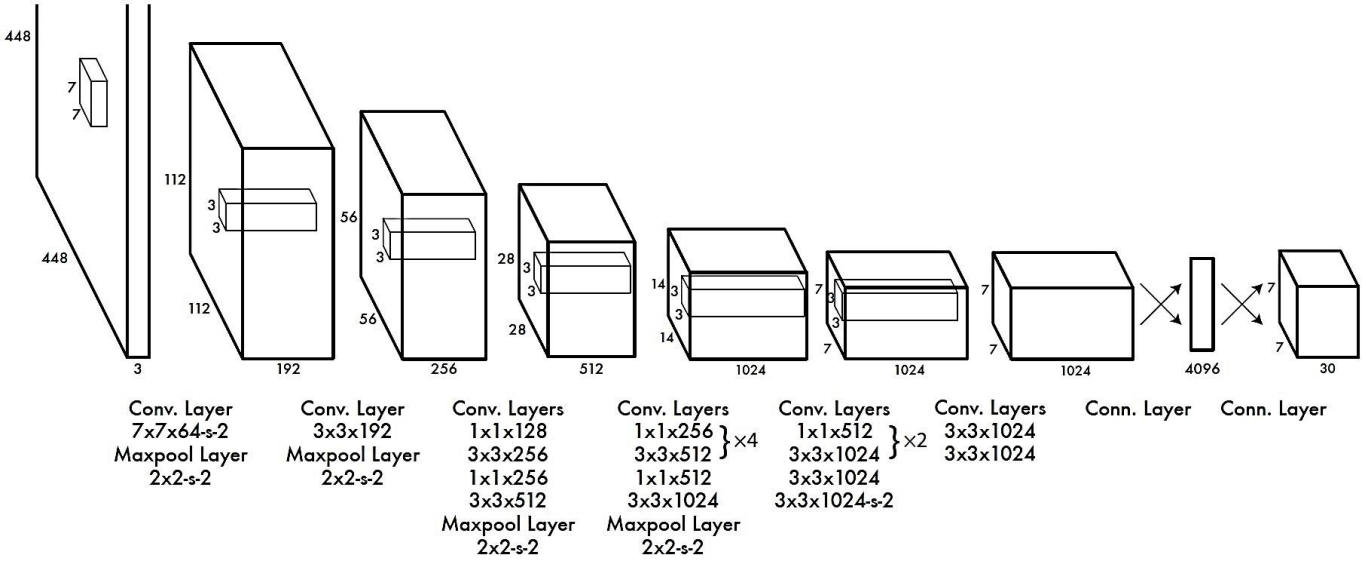
- Each cell needs to predict (B*5+C) values.
- If the input image is divided into an S×S grid, then the final predicted value is S×S×(B*5+C).
- YOLOv1 uses the PASCAL VOC dataset for training. The PASCAL VOC dataset has 20 categories. If we set S=7 and B=2, then the final prediction result is a tensor of size 7×7×30.





YOLO Network Structure

- YOLO uses convolutional networks to extract features, and then fully connected layers to obtain predicted values.
- The network structure is based on the GooLeNet model, containing 24 convolutional layers and 2 fully connected layers.
- For the convolutional layers, 1x1 convolutions are primarily used for channel reduction, followed by 3x3 convolutions.
- The Leaky ReLU activation function, max(x, 0.1x), is used for both the convolutional and fully connected layers.
- The final layer, however, uses a linear activation function.

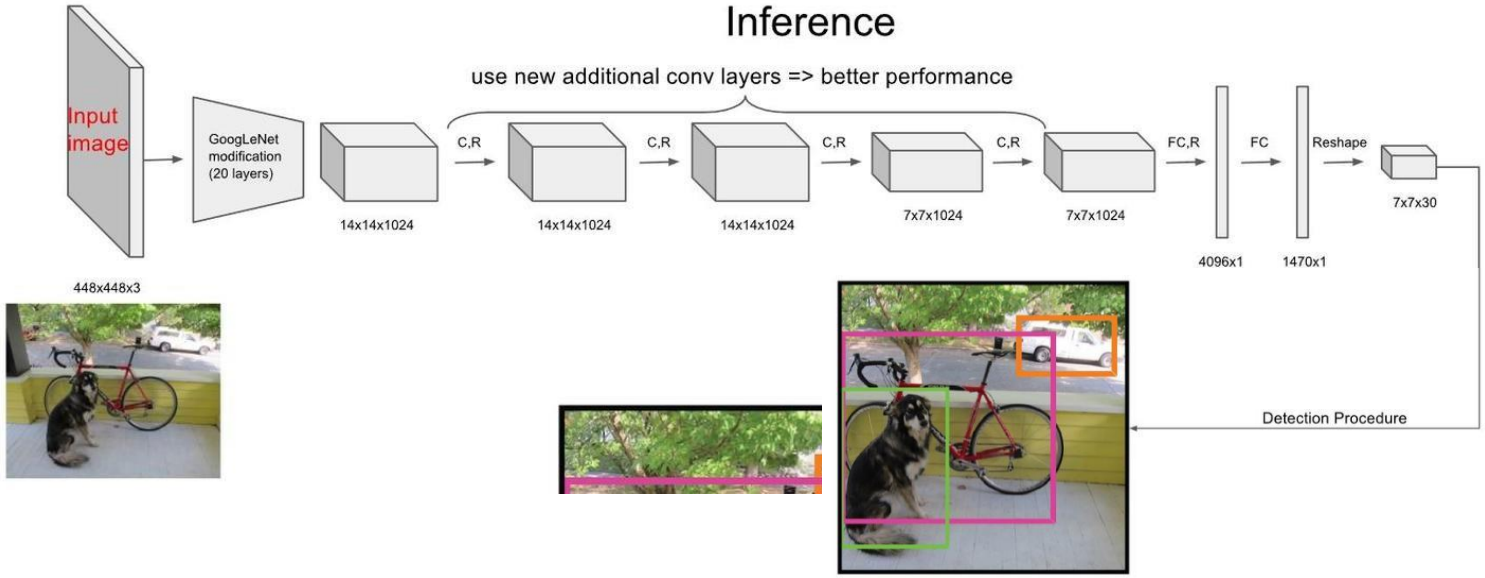


YOLO Network Structure

- YOLOv1 Network Structure:
  - Input: 448×448×3 image.
  - Backbone Network: 24 convolutional layers + 2 fully connected layers.
  - Output: 7×7×30 tensor (each grid predicts 2 bounding boxes and 20 class probabilities).
- Improvements in YOLOv2/v3:
  - Uses Darknet-19/Darknet-53 as the backbone network.
  - Introduces anchor boxes and multi-scale prediction.

YOLO Network Training

- Before training, the network was pre-trained on ImageNet. The pre-trained classification model used the first 20 convolutional layers shown in Figure 8, followed by an average-pooling layer and a fully connected layer.
- After pre-training, four randomly initialized convolutional layers and two fully connected layers were added on top of the 20 pre-trained convolutional layers.
- Since detection tasks generally require higher-resolution images, the network input was increased from 224x224 to 448x448.



YOLO Loss Function

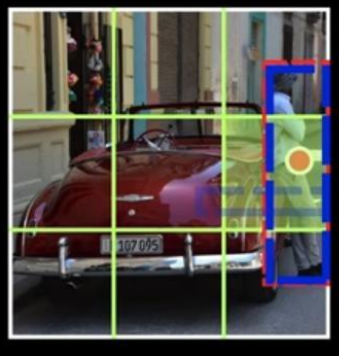
1. Bounding box coordinate loss
2. Confidence loss
3. Class loss

Formula:
Total Loss = Coordinate Loss + Confidence Loss + Class Loss

$$\left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2\right]$$

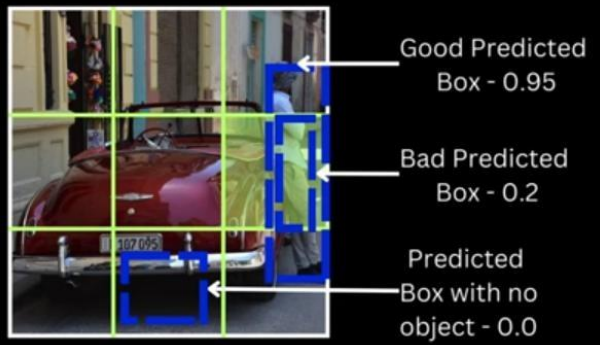
$$\left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i}\right)^2\right]$$

Localization Loss



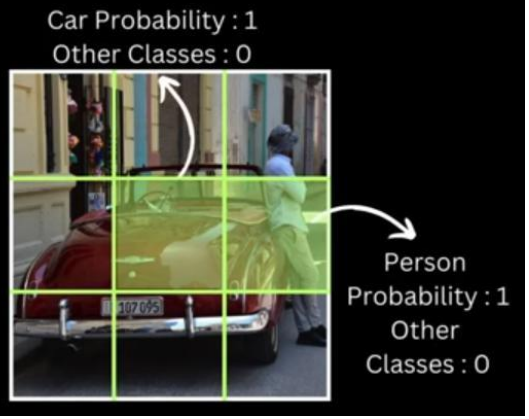
$$\left(C_i - \hat{C}_i\right)^2$$

Confidence Loss



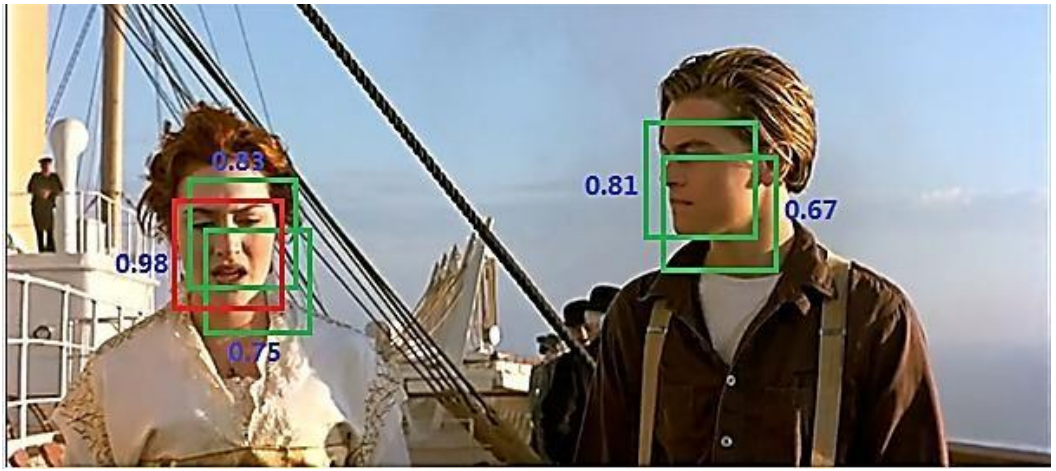
$$\sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

Classification Loss



YOLO Network Prediction

- Non-Maximum Suppression (NMS) is an algorithm that primarily addresses the problem of a target being detected multiple times.
- As shown in the face detection example, the face is detected multiple times, but we ultimately want to output only the best predicted bounding box. For example, for the woman on the left, we only want the red detection result.



- The NMS algorithm is used to achieve this effect: First, find the box with the highest confidence among all detected boxes. Then, calculate the Intersection over Union (IoU) between the box and the remaining boxes. If the IoU value is greater than a certain threshold (too high overlap), the box is removed.
- The above process is repeated for the remaining detected boxes until all detected boxes have been processed.

YOLO System Implementation

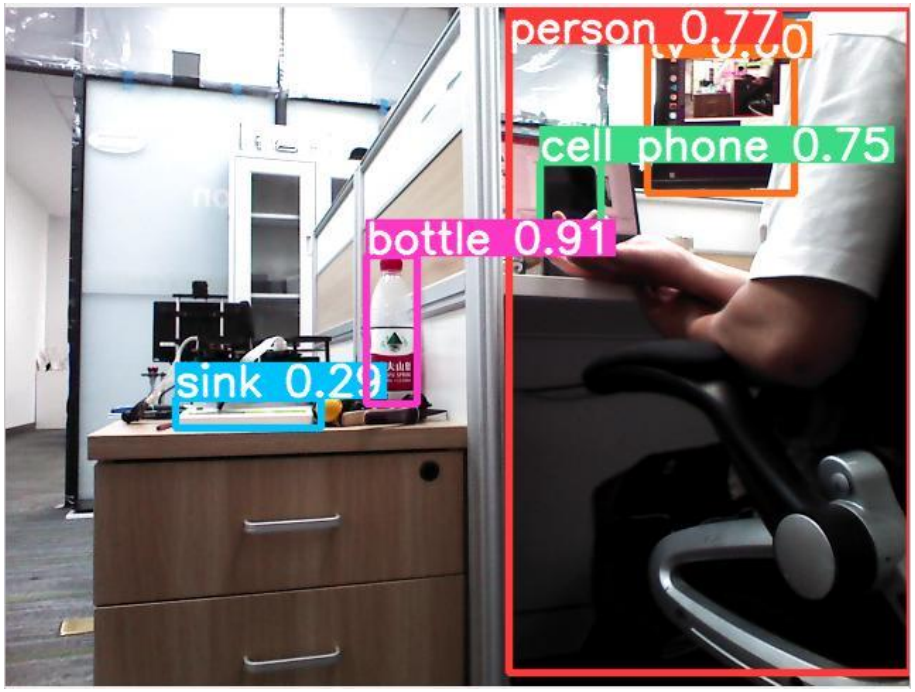
Launch camera

- $ roslaunch astra_camera astra.launch

Launch YOLO system

- $ roslaunch jupiterobot2_vision_yolov5 object_detection.launch

Code:
https://github.com/ultralytics/



YOLO Version Development

- YOLOv1 (2016):
  - First proposed, fast but with lower accuracy.
- YOLOv2 (2017):
  - Introduced Anchor Boxes, improving accuracy.
- YOLOv3 (2018):
  - Multi-scale prediction, better small object detection.
- YOLOv4 (2020):
  - Introduced advanced technologies such as CSPDarknet53 and PANet.
- YOLOv5 (2020):
  - Implemented in PyTorch, easy to use and deploy.
- YOLOv6/v7/v8:
  - Further optimized speed and accuracy.

YOLO Application Scenarios

- Autonomous Driving:
  - Detecting pedestrians, vehicles, and traffic signs.
- Security Monitoring:
  - Real-time monitoring and abnormal behavior detection.
- Retail Industry:
  - Inventory management and customer behavior analysis.
- Healthcare:
  - Medical image analysis and disease diagnosis.

YOLO Advantages and Limitations

- Advantages:
  - High speed, suitable for real-time applications.
  - Global context information reduces false detections.
- Limitations:
  - Limited ability to detect small and densely packed targets.
  - Requires a large amount of labeled data for training.

References

- Joseph Redmon, et al. "You Only Look Once: Unified, Real-Time Object Detection." (2016)
- Joseph Redmon, Ali Farhadi. "YOLO9000: Better, Faster, Stronger." (2017)
- Joseph Redmon, Ali Farhadi. "YOLOv3: An Incremental Improvement." (2018)
- Alexey Bochkovskiy, et al. "YOLOv4: Optimal Speed and Accuracy of Object Detection." (2020)

YOLO Object Detection

- Bring up
  - $ roslaunch astra_camera **astra.launch**
- YOLO Object Detection
  - $ roslaunch jupiterobot2_vision_yolov5 **object_detection.launch**

Train YOLOv5 on a Custom Dataset

1. Setup YOLOv5.
2. Prepare the **dataset** in YOLO format.
3. Configure the **data.yaml** file with dataset paths.
4. Start training using **train.py**.

References:

- https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/
- https://www.digitalocean.com/community/tutorials/train-yolov5-custom-data
- Ultralytics Platform: https://platform.ultralytics.com/



MINI PROJECTS

Mini Projects

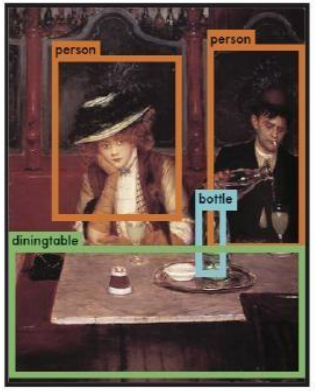
- Design a practical application for a service duty in a specific environment.
- Based on the technologies introduced in this course, design the speech and vision interactions of the system with its audience and environment during service.

Project Presentations

- Prepare a set of project presentation slides.
  - Introduction
  - System Design
  - Speech and Vision Interactions
- Present the system design and the interactions by demonstrating the scenario with the system.





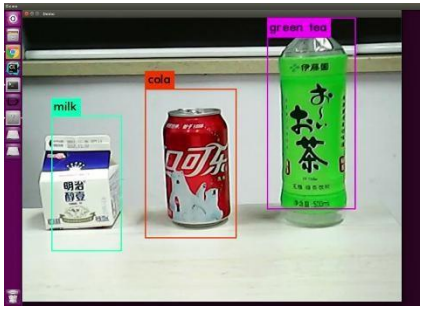


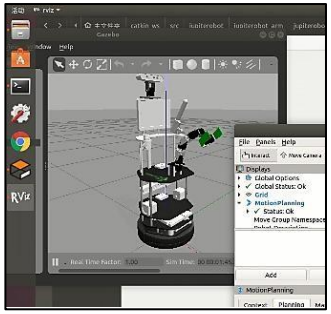
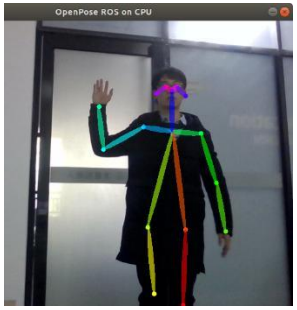


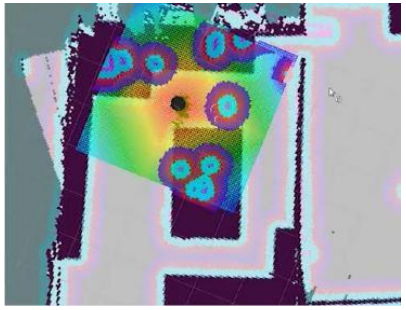
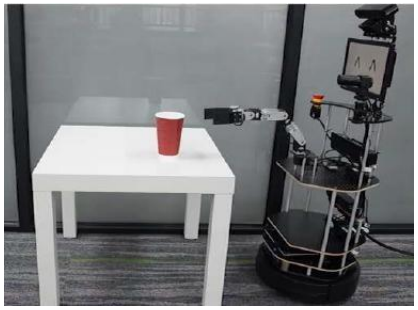


Machine Learning

PRACTICAL APPLICATIONS



UNIVERSITY OF JINAN

08 – 10 Nov 2026 | Jeffrey Tan